

Design Patterns: A Complete Guide

A structured and beginner-friendly reference to the three major categories of software design patterns — Creational, Structural, and Behavioral.

Design patterns are reusable solutions to commonly occurring problems in software design. They are not code — they are blueprints or templates that guide how to structure your code to solve a problem efficiently and elegantly.

Table of Contents

- [1. Creational Design Patterns](#)
 - [Singleton](#)
 - [Factory Method](#)
 - [Abstract Factory](#)
 - [Builder](#)
 - [Prototype](#)
 - [2. Structural Design Patterns](#)
 - [Adapter](#)
 - [Bridge](#)
 - [Composite](#)
 - [Decorator](#)
 - [Facade](#)
 - [Flyweight](#)
 - [Proxy](#)
 - [3. Behavioral Design Patterns](#)
 - [Chain of Responsibility](#)
 - [Command](#)
 - [Iterator](#)
 - [Mediator](#)
 - [Memento](#)
 - [Observer](#)
 - [State](#)
 - [Strategy](#)
 - [Template Method](#)
 - [Visitor](#)
-

1. Creational Design Patterns

Overview

Creational patterns deal with **how objects are created**. They abstract the instantiation process, making a system independent of how its objects are created, composed, and represented. These patterns give you more flexibility in deciding which objects need to be created for a given situation.

Singleton

Description: Ensures that a class has only **one instance** throughout the lifetime of an application and provides a global access point to that instance.

- **Why it is used:** To control access to shared resources such as a configuration manager, logger, or database connection — where having multiple instances would cause conflicts or waste resources.
 - **Where it is commonly used:** Logging systems, application configuration managers, thread pools, caching systems.
-

Code Example (JAVASCRIPT)

```
/*
 * SINGLETON DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Singleton pattern ensures a class has only one instance and provides
a global point of access to that instance.
 *
 * WHY DOES IT EXIST?
 * - Some resources should only exist once (database connections, logging
services, configuration managers)
 * - Prevents multiple instances from causing conflicts or inconsistent
state
 * - Provides controlled access to the sole instance
 *
 * WHEN TO USE IT?
 * - When exactly one instance of a class is needed to coordinate actions
across the system
 * - When the sole instance should be extensible by subclassing, and
clients should be able to use an extended instance without modifying their
code
 * - Common use cases: database connection pools, logging services,
configuration managers, thread pools, caches
 *
 * PROS:
 * - Controlled access to the sole instance
 * - Reduced namespace pollution (compared to global variables)
 * - Can be subclassed
 * - Can be lazy-loaded (created only when first needed)
 *
 * CONS:
 * - Violates Single Responsibility Principle (handles both its business
logic and instance creation)
 * - Can hide dependencies (making code harder to test)
 * - Global state can lead to unexpected behavior in multi-threaded
environments
 * - Difficult to unit test due to global state
 *
 * REAL-WORLD ANALOGY:
 * Think of a government - there's only one president or prime minister at
```

a time. Everyone who needs to interact with the head of state goes through the same single instance. You can't have two presidents simultaneously making conflicting decisions.

```
*/

// Example 1: Basic Singleton using static property
class DatabaseConnection {
  // Private static property to hold the single instance
  private static instance: DatabaseConnection;

  // Private constructor to prevent instantiation from outside
  private constructor(private isConnected: boolean = false) {
    console.log('DatabaseConnection: Initializing connection...');
    this.isConnected = true;
  }

  // Static method to get the single instance
  public static getInstance(): DatabaseConnection {
    if (!DatabaseConnection.instance) {
      DatabaseConnection.instance = new DatabaseConnection();
    }
    return DatabaseConnection.instance;
  }

  // Business logic methods
  public executeQuery(query: string): void {
    if (this.isConnected) {
      console.log(`Executing query: ${query}`);
    } else {
      console.log('Error: Not connected to database');
    }
  }

  public getConnectionStatus(): boolean {
    return this.isConnected;
  }
}

// Example 2: Singleton with lazy initialization using getter
class Logger {
  private static _instance: Logger;
  private logCount: number = 0;

  // Private constructor
  private constructor() {
    console.log('Logger: Initializing logger...');
  }

  // Static getter for lazy initialization
  public static get instance(): Logger {
    if (!Logger._instance) {
      Logger._instance = new Logger();
    }
    return Logger._instance;
  }
}
```

```

    }

    public log(message: string): void {
        this.logCount++;
        console.log(`[LOG #${this.logCount}] ${message}`);
    }

    public getLogCount(): number {
        return this.logCount;
    }
}

// Example 3: Singleton with initialization parameters
class ConfigurationManager {
    private static instance: ConfigurationManager;
    private config: Map<string, string>;

    // Private constructor with parameters
    private constructor(initialConfig?: Map<string, string>) {
        console.log('ConfigurationManager: Initializing...');
        this.config = initialConfig || new Map();
    }

    // Static method with optional initialization parameters
    public static getInstance(initialConfig?: Map<string, string>):
    ConfigurationManager {
        if (!ConfigurationManager.instance) {
            ConfigurationManager.instance = new
            ConfigurationManager(initialConfig);
        }
        return ConfigurationManager.instance;
    }

    public set(key: string, value: string): void {
        this.config.set(key, value);
    }

    public get(key: string): string | undefined {
        return this.config.get(key);
    }

    public displayConfig(): void {
        console.log('Current Configuration:');
        this.config.forEach((value, key) => {
            console.log(`  ${key}: ${value}`);
        });
    }
}

// Example 4: Singleton using module pattern (TypeScript-specific)
// This leverages TypeScript's module system to create a singleton
const SingletonService = (() => {
    class Service {
        private data: string[] = [];
    }

```

```

    // Constructor is public because the class itself is private to this
    scope
    constructor() {
        console.log('Service: Initializing...');
    }

    public addData(item: string): void {
        this.data.push(item);
    }

    public getData(): string[] {
        return [...this.data];
    }
}

let instance: Service | null = null;

return {
    getInstance: (): Service => {
        if (!instance) {
            instance = new Service();
        }
        return instance;
    }
};
})();

// Example 5: Async Thread-safe Singleton
// In environments where multiple async calls might trigger initialization.
// The "System Design" way in JS/TS is to use a Promise as a lock.
class AsyncSingleton {
    private static instance: AsyncSingleton | null = null;
    private static initPromise: Promise<AsyncSingleton> | null = null;

    private constructor() {
        console.log('AsyncSingleton: Instance created');
    }

    public static async getInstance(): Promise<AsyncSingleton> {
        // 1. If instance already exists, return it immediately
        if (AsyncSingleton.instance) return AsyncSingleton.instance;

        // 2. If initialization is already in progress, return the existing
        promise
        if (AsyncSingleton.initPromise) {
            console.log('AsyncSingleton: Initialization already in progress,
waiting...');
            return AsyncSingleton.initPromise;
        }

        // 3. Start initialization and store the promise (the "Lock")
        AsyncSingleton.initPromise = (async () => {
            console.log('AsyncSingleton: Starting heavy async

```

```

    initialization...');
    await new Promise(resolve => setTimeout(resolve, 100)); // Simulate
delay
    AsyncSingleton.instance = new AsyncSingleton();
    return AsyncSingleton.instance;
  })();

  return AsyncSingleton.initPromise;
}
}

// Example 6: Bypassing Singleton
// In TypeScript, 'private' is a compile-time check. At runtime, JavaScript
// reflection can still access and call the constructor.
class BypassableSingleton {
  private static instance: BypassableSingleton;
  private constructor() {
    console.log('BypassableSingleton: Instance created');
  }
  public static getInstance(): BypassableSingleton {
    if (!BypassableSingleton.instance) BypassableSingleton.instance = new
BypassableSingleton();
    return BypassableSingleton.instance;
  }
}

// Demo code
function demoSingleton(): void {
  console.log('=== Singleton Pattern Demo ===\n');

  // Example 1: Basic Singleton
  console.log('--- DatabaseConnection Example ---');
  const db1 = DatabaseConnection.getInstance();
  db1.executeQuery('SELECT * FROM users');

  const db2 = DatabaseConnection.getInstance();
  db2.executeQuery('SELECT * FROM products');

  console.log(`Same instance? ${db1 === db2 ? 'Yes' : 'No'}\n`);

  // Example 2: Singleton with getter
  console.log('--- Logger Example ---');
  Logger.instance.log('Application started');
  Logger.instance.log('User logged in');
  Logger.instance.log('Processing request');
  console.log(`Total logs: ${Logger.instance.getLogCount()}\n`);

  // Example 3: Singleton with initialization parameters
  console.log('--- ConfigurationManager Example ---');
  const initialConfig = new Map<string, string>([
    ['apiUrl', 'https://api.example.com'],
    ['timeout', '5000']
  ]);
  const config = ConfigurationManager.getInstance(initialConfig);

```

```
config.displayConfig();

config.set('apiKey', 'abc123');
config.displayConfig();

const config2 = ConfigurationManager.getInstance();
console.log(`Same instance? ${config === config2 ? 'Yes' : 'No'}\n`);

// Example 4: Module pattern singleton
console.log('--- Module Pattern Singleton ---');
const service1 = SingletonService.getInstance();
service1.addData('Item 1');
service1.addData('Item 2');

const service2 = SingletonService.getInstance();
service2.addData('Item 3');

console.log('Service data:', service2.getData());
console.log(`Same instance? ${service1 === service2 ? 'Yes' : 'No'}\n`);

// Example 5: Async Thread-safe Singleton
console.log('--- AsyncSingleton Example ---');
Promise.all([
  AsyncSingleton.getInstance(),
  AsyncSingleton.getInstance(),
  AsyncSingleton.getInstance()
]).then((instances) => {
  console.log(`All instances are same: ${instances.every(i => i === instances[0])}`);
});

// Example 6: Bypassing Singleton
console.log('--- Bypassing Singleton Example ---');
const b1 = BypassableSingleton.getInstance();

// Bypassing using Reflect.construct (even if constructor is private)
try {
  const b2 = Reflect.construct(BypassableSingleton, []) as BypassableSingleton;
  console.log(`Bypass successful! Same instance? ${b1 === b2 ? 'Yes' : 'No'}\n`);
} catch (e) {
  console.log('Bypass failed:', e);
}

// Run the demo
demoSingleton();
```

Factory Method

Description: Defines an interface for creating an object, but **lets subclasses decide** which class to instantiate. The parent class defers object creation to its child classes.

- **Why it is used:** To promote loose coupling by eliminating the need to bind application-specific classes into your code. The creation logic is centralized and easy to extend.
- **Where it is commonly used:** UI frameworks (creating different buttons for different OS), plugin systems, document generators.

Code Example (JAVASCRIPT)

```
/*
 * FACTORY METHOD DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Factory Method pattern defines an interface for creating an object
but lets subclasses decide which class to instantiate.
 * It delegates the instantiation logic to child classes.
 *
 * WHY DOES IT EXIST?
 * - Decouples the creation of objects from their usage
 * - Allows a class to defer instantiation to subclasses
 * - Provides a way to extend the creation logic without modifying existing
code (Open/Closed Principle)
 * - Eliminates the need to bind application-specific classes into your
code
 *
 * WHEN TO USE IT?
 * - When a class can't anticipate the class of objects it must create
 * - When a class wants its subclasses to specify the objects it creates
 * - When classes delegate responsibility to one of several helper
subclasses, and you want to localize the knowledge of which helper subclass
is the delegate
 * - Common use cases: document creation, UI components, payment
processing, logging frameworks
 *
 * PROS:
 * - Avoids tight coupling between the creator and the concrete products
 * - Follows Open/Closed Principle (easy to introduce new products without
changing existing code)
 * - Single Responsibility Principle (product creation code is in one
place)
 * - Can be combined with other patterns (e.g., Singleton, Prototype)
 *
 * CONS:
 * - Can make the code more complex by introducing many subclasses
 * - May require creating a parallel inheritance hierarchy for creators and
products
 * - Can lead to excessive use of inheritance
 *
 * REAL-WORLD ANALOGY:
 * Think of a car manufacturing plant. The plant (factory) knows how to
```

build cars, but it doesn't know what specific type of car to build until an order comes in. The plant has a method to build cars, but the specific car model (sedan, SUV, truck) is determined by the order or by specialized assembly lines (subclasses).

*/

// Product interface - defines the common interface for all products

```
interface IDocument {  
  open(): void;  
  close(): void;  
  save(): void;  
  getType(): string;  
}
```

// Concrete Product 1: Text Document

```
class TextDocument implements IDocument {  
  open(): void {  
    console.log('Opening Text Document in text editor...');  
  }  
  
  close(): void {  
    console.log('Closing Text Document...');  
  }  
  
  save(): void {  
    console.log('Saving Text Document as .txt file...');  
  }  
  
  getType(): string {  
    return 'Text Document';  
  }  
}
```

// Concrete Product 2: PDF Document

```
class PDFDocument implements IDocument {  
  open(): void {  
    console.log('Opening PDF Document in PDF viewer...');  
  }  
  
  close(): void {  
    console.log('Closing PDF Document...');  
  }  
  
  save(): void {  
    console.log('Saving PDF Document as .pdf file...');  
  }  
  
  getType(): string {  
    return 'PDF Document';  
  }  
}
```

// Concrete Product 3: HTML Document

```
class HtmlDocument implements IDocument {
```

```
open(): void {
  console.log('Opening HTML Document in web browser...');
}

close(): void {
  console.log('Closing HTML Document...');
}

save(): void {
  console.log('Saving HTML Document as .html file...');
}

getType(): string {
  return 'HTML Document';
}
}

// Creator abstract class - declares the factory method
abstract class DocumentApplication {
  // Factory method - to be implemented by subclasses
  // This is the core of the pattern: subclasses decide what to create
  protected abstract createDocument(): IDocument;

  // Business logic that uses the product
  // This method doesn't need to know which specific document is created
  public newDocument(): void {
    const doc = this.createDocument();
    console.log(`Created: ${doc.getType()}`);
    doc.open();
    doc.save();
    doc.close();
  }
}

// Concrete Creator 1: Text Editor Application
class TextEditor extends DocumentApplication {
  protected createDocument(): IDocument {
    return new TextDocument();
  }
}

// Concrete Creator 2: PDF Viewer Application
class PDFViewer extends DocumentApplication {
  protected createDocument(): IDocument {
    return new PDFDocument();
  }
}

// Concrete Creator 3: Web Browser Application
class WebBrowser extends DocumentApplication {
  protected createDocument(): IDocument {
    return new HtmlDocument();
  }
}
```

```
// Alternative implementation: Parameterized Factory Method
// This allows a single creator to create multiple types of products
enum DocType {
  TEXT,
  PDF,
  HTML
}

class DocumentFactory {
  public static createDocument(type: DocType): IDocument {
    switch (type) {
      case DocType.TEXT:
        return new TextDocument();
      case DocType.PDF:
        return new PDFDocument();
      case DocType.HTML:
        return new HtmlDocument();
      default:
        throw new Error('Unknown document type');
    }
  }
}

// Example: Payment Processing Factory
interface PaymentProcessor {
  processPayment(amount: number): void;
  refundPayment(amount: number): void;
}

class CreditCardProcessor implements PaymentProcessor {
  processPayment(amount: number): void {
    console.log(`Processing credit card payment of $$${amount}`);
  }

  refundPayment(amount: number): void {
    console.log(`Refunding credit card payment of $$${amount}`);
  }
}

class PayPalProcessor implements PaymentProcessor {
  processPayment(amount: number): void {
    console.log(`Processing PayPal payment of $$${amount}`);
  }

  refundPayment(amount: number): void {
    console.log(`Refunding PayPal payment of $$${amount}`);
  }
}

class CryptoProcessor implements PaymentProcessor {
  processPayment(amount: number): void {
    console.log(`Processing cryptocurrency payment of $$${amount}`);
  }
}
```

```
refundPayment(amount: number): void {
    console.log(`Refunding cryptocurrency payment of ${amount}`);
}

enum PaymentMethod {
    CREDIT_CARD,
    PAYPAL,
    CRYPTO
}

class PaymentFactory {
    public static createProcessor(method: PaymentMethod): PaymentProcessor {
        switch (method) {
            case PaymentMethod.CREDIT_CARD:
                return new CreditCardProcessor();
            case PaymentMethod.PAYPAL:
                return new PayPalProcessor();
            case PaymentMethod.CRYPTO:
                return new CryptoProcessor();
            default:
                throw new Error('Unknown payment method');
        }
    }
}

// Demo code
function demoFactoryMethod(): void {
    console.log('=== Factory Method Pattern Demo ===\n');

    // Using inheritance-based Factory Method
    console.log('--- Inheritance-based Factory Method ---');

    const textEditor = new TextEditor();
    textEditor.newDocument();
    console.log();

    const pdfViewer = new PDFViewer();
    pdfViewer.newDocument();
    console.log();

    const webBrowser = new WebBrowser();
    webBrowser.newDocument();
    console.log();

    // Using parameterized Factory Method
    console.log('--- Parameterized Factory Method ---');

    const textDoc = DocumentFactory.createDocument(DocType.TEXT);
    textDoc.open();
    console.log();

    const pdfDoc = DocumentFactory.createDocument(DocType.PDF);
```

```
pdfDoc.open();
console.log();

const htmlDoc = DocumentFactory.createDocument(DocType.HTML);
htmlDoc.open();
console.log();

// Payment Processing Example
console.log('--- Payment Processing Example ---');

const creditCardProcessor =
PaymentFactory.createProcessor(PaymentMethod.CREDIT_CARD);
creditCardProcessor.processPayment(100);
creditCardProcessor.refundPayment(50);
console.log();

const paypalProcessor =
PaymentFactory.createProcessor(PaymentMethod.PAYPAL);
paypalProcessor.processPayment(75);
console.log();

const cryptoProcessor =
PaymentFactory.createProcessor(PaymentMethod.CRYPTO);
cryptoProcessor.processPayment(200);
}

// Run the demo
demoFactoryMethod();
```

Abstract Factory

Description: Provides an interface for creating **families of related or dependent objects** without specifying their concrete classes.

- **Why it is used:** To ensure that a set of related objects are used together without mixing incompatible combinations. It's like a factory of factories.
- **Where it is commonly used:** Cross-platform UI toolkits (e.g., Windows vs. macOS components), database drivers, theme systems.

Code Example (JAVASCRIPT)

```
/*
 * ABSTRACT FACTORY DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Abstract Factory pattern provides an interface for creating families
of related or dependent objects without specifying their concrete classes.
 * It's like a factory of factories.
 *
 * WHY DOES IT EXIST?
```

```

* - When you need to create families of related objects (e.g., UI
components for different operating systems)
* - When you want to ensure that created objects are compatible with each
other
* - When you want to provide a library of products without exposing their
implementation details
* - When you need to support multiple themes or platforms
*
* WHEN TO USE IT?
* - When a system should be independent of how its products are created,
composed, and represented
* - When a system should be configured with one of multiple families of
products
* - When the family of related product objects is designed to be used
together, and you need to enforce this constraint
* - When you want to provide a class library of products, and you want to
reveal just their interfaces, not their implementations
* - Common use cases: cross-platform UI components, database connectors
for different DBMS, game development (different themes)
*
* PROS:
* - Ensures that products from the same family are compatible
* - Hides the concrete classes from the client code
* - Easy to introduce new product families without modifying existing code
(Open/Closed Principle)
* - Promotes consistency among products
*
* CONS:
* - Can be complex to implement due to many interfaces and classes
* - Adding new products to existing families requires modifying all
factory interfaces
* - Can lead to excessive abstraction and indirection
* - May be overkill for simple scenarios
*
* REAL-WORLD ANALOGY:
* Think of a furniture factory that produces different styles of furniture
(Modern, Victorian, Rustic). Each style has its own family of products
(chairs, tables, sofas). If you order a Modern chair, you want it to match
with a Modern table. The abstract factory ensures that all furniture pieces
you order belong to the same style family.
*/

// Abstract Product A: Button interface
interface Button {
    click(): void;
    render(): void;
    getName(): string;
}

// Abstract Product B: Checkbox interface
interface Checkbox {
    check(): void;
    render(): void;
    getName(): string;
}

```

```
}

// Concrete Product A1: Windows Button
class WindowsButton implements Button {
  click(): void {
    console.log('Windows button clicked!');
  }

  render(): void {
    console.log('Rendering Windows-style button...');
  }

  getName(): string {
    return 'Windows Button';
  }
}

// Concrete Product B1: Windows Checkbox
class WindowsCheckbox implements Checkbox {
  check(): void {
    console.log('Windows checkbox checked!');
  }

  render(): void {
    console.log('Rendering Windows-style checkbox...');
  }

  getName(): string {
    return 'Windows Checkbox';
  }
}

// Concrete Product A2: Mac Button
class MacButton implements Button {
  click(): void {
    console.log('Mac button clicked!');
  }

  render(): void {
    console.log('Rendering Mac-style button...');
  }

  getName(): string {
    return 'Mac Button';
  }
}

// Concrete Product B2: Mac Checkbox
class MacCheckbox implements Checkbox {
  check(): void {
    console.log('Mac checkbox checked!');
  }

  render(): void {
```

```
        console.log('Rendering Mac-style checkbox...');
    }

    getName(): string {
        return 'Mac Checkbox';
    }
}

// Abstract Factory interface - declares methods for creating each abstract
// product
interface GUIFactory {
    createButton(): Button;
    createCheckbox(): Checkbox;
}

// Concrete Factory 1: Windows Factory
// Creates Windows-specific UI components
class WindowsFactory implements GUIFactory {
    createButton(): Button {
        return new WindowsButton();
    }

    createCheckbox(): Checkbox {
        return new WindowsCheckbox();
    }
}

// Concrete Factory 2: Mac Factory
// Creates Mac-specific UI components
class MacFactory implements GUIFactory {
    createButton(): Button {
        return new MacButton();
    }

    createCheckbox(): Checkbox {
        return new MacCheckbox();
    }
}

// Client class that uses the abstract factory
// The client doesn't know which concrete factory is being used
class GUIApplication {
    private button: Button;
    private checkbox: Checkbox;

    constructor(factory: GUIFactory) {
        this.button = factory.createButton();
        this.checkbox = factory.createCheckbox();
    }

    // Business logic that uses the products
    public renderUI(): void {
        console.log('Rendering UI components...');
        console.log(`Button: ${this.button.getName()}`);
    }
}
```

```
        console.log(`Checkbox: ${this.checkbox.getName()}`);

        this.button.render();
        this.checkbox.render();
    }

    public interact(): void {
        console.log(`\nUser interaction...`);
        this.button.click();
        this.checkbox.check();
    }
}

// Example: Database Abstract Factory
// Shows how Abstract Factory can be used for database connections

// Abstract Product: Database Connection
interface IDatabaseConnection {
    connect(): void;
    disconnect(): void;
    executeQuery(query: string): void;
}

// Abstract Product: Database Command
interface DatabaseCommand {
    execute(): void;
}

// Concrete Products: MySQL
class MySQLConnection implements IDatabaseConnection {
    connect(): void {
        console.log('Connecting to MySQL database...');
    }

    disconnect(): void {
        console.log('Disconnecting from MySQL database...');
    }

    executeQuery(query: string): void {
        console.log(`Executing MySQL query: ${query}`);
    }
}

class MySQLCommand implements DatabaseCommand {
    constructor(private command: string) {}

    execute(): void {
        console.log(`Executing MySQL command: ${this.command}`);
    }
}

// Concrete Products: PostgreSQL
class PostgreSQLConnection implements IDatabaseConnection {
    connect(): void {
```

```
        console.log('Connecting to PostgreSQL database...');
    }

    disconnect(): void {
        console.log('Disconnecting from PostgreSQL database...');
    }

    executeQuery(query: string): void {
        console.log(`Executing PostgreSQL query: ${query}`);
    }
}

class PostgreSQLCommand implements DatabaseCommand {
    constructor(private command: string) {}

    execute(): void {
        console.log(`Executing PostgreSQL command: ${this.command}`);
    }
}

// Abstract Factory for Database
interface DatabaseFactory {
    createConnection(): IDatabaseConnection;
    createCommand(cmd: string): DatabaseCommand;
}

// Concrete Factory: MySQL
class MySQLFactory implements DatabaseFactory {
    createConnection(): IDatabaseConnection {
        return new MySQLConnection();
    }

    createCommand(cmd: string): DatabaseCommand {
        return new MySQLCommand(cmd);
    }
}

// Concrete Factory: PostgreSQL
class PostgreSQLFactory implements DatabaseFactory {
    createConnection(): IDatabaseConnection {
        return new PostgreSQLConnection();
    }

    createCommand(cmd: string): DatabaseCommand {
        return new PostgreSQLCommand(cmd);
    }
}

// Demo code
function demoAbstractFactory(): void {
    console.log('=== Abstract Factory Pattern Demo ===\n');

    // Example 1: GUI Components
    console.log('--- GUI Components Example ---');
```

```
// Create Windows UI
console.log('\nCreating Windows UI:');
const windowsFactory = new WindowsFactory();
const windowsApp = new GUIApplication(windowsFactory);
windowsApp.renderUI();
windowsApp.interact();

// Create Mac UI
console.log('\nCreating Mac UI:');
const macFactory = new MacFactory();
const macApp = new GUIApplication(macFactory);
macApp.renderUI();
macApp.interact();

// Example 2: Database Connections
console.log('\n\n--- Database Connections Example ---');

// MySQL
console.log('\nUsing MySQL:');
const mysqlFactory = new MySQLFactory();
const mysqlConnection = mysqlFactory.createConnection();
const mysqlCommand = mysqlFactory.createCommand('CREATE TABLE users (id
INT)');

mysqlConnection.connect();
mysqlConnection.executeQuery('SELECT * FROM users');
mysqlCommand.execute();
mysqlConnection.disconnect();

// PostgreSQL
console.log('\nUsing PostgreSQL:');
const postgresFactory = new PostgreSQLFactory();
const postgresConnection = postgresFactory.createConnection();
const postgresCommand = postgresFactory.createCommand('CREATE TABLE
products (id INT)');

postgresConnection.connect();
postgresConnection.executeQuery('SELECT * FROM products');
postgresCommand.execute();
postgresConnection.disconnect();
}

// Run the demo
demoAbstractFactory();
```

Builder

Description: Separates the **construction of a complex object** from its representation, allowing the same construction process to create different representations.

- **Why it is used:** To construct objects step-by-step, especially when an object requires many configuration options or when different representations of a product are needed.
 - **Where it is commonly used:** Building complex query objects, constructing HTML/XML documents, configuring network requests.
-

Code Example (JAVASCRIPT)

```
/*
 * BUILDER DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Builder pattern separates the construction of a complex object from
its representation, allowing the same construction process to create
different representations.
 * It constructs complex objects step by step.
 *
 * WHY DOES IT EXIST?
 * - When you need to create complex objects with many optional parameters
 * - When you want to separate the construction logic from the object's
representation
 * - When you need to create objects in a step-by-step fashion
 * - When you want to provide different representations for the same
construction process
 *
 * WHEN TO USE IT?
 * - When the algorithm for creating a complex object should be independent
of the parts that make up the object and how they're assembled
 * - When the construction process must allow different representations for
the object that's constructed
 * - When you want to construct objects with many optional parameters
without using a telescoping constructor
 * - Common use cases: building complex documents, SQL query builders,
HTML/XML generators, configuration objects
 *
 * PROS:
 * - Allows you to construct complex objects step by step
 * - Provides fine-grained control over the construction process
 * - Can reuse the same construction process to create different
representations
 * - Single Responsibility Principle (complex construction code is isolated
from business logic)
 * - More readable than telescoping constructors
 *
 * CONS:
 * - Increases overall code complexity
 * - Requires creating a separate Builder class for each product
 * - May not be necessary for simple objects
 * - Can lead to code duplication if not carefully designed
 *
 * REAL-WORLD ANALOGY:
 * Think of ordering a custom pizza. You don't just say "give me a pizza" -
```

you specify the crust, sauce, cheese, and toppings step by step. The pizza builder (chef) takes your specifications and constructs the pizza. You can use the same process to create different types of pizzas (vegetarian, meat lover's, etc.) by choosing different options at each step.

```
*/
```

```
// Product class - the complex object being built
class House {
  private foundation: string = '';
  private structure: string = '';
  private roof: string = '';
  private interior: string = '';
  private hasGarage: boolean = false;
  private hasGarden: boolean = false;
  private hasSwimmingPool: boolean = false;

  // Setter methods for builder to use
  public setFoundation(f: string): void {
    this.foundation = f;
  }

  public setStructure(s: string): void {
    this.structure = s;
  }

  public setRoof(r: string): void {
    this.roof = r;
  }

  public setInterior(i: string): void {
    this.interior = i;
  }

  public setGarage(g: boolean): void {
    this.hasGarage = g;
  }

  public setGarden(g: boolean): void {
    this.hasGarden = g;
  }

  public setSwimmingPool(s: boolean): void {
    this.hasSwimmingPool = s;
  }

  // Display the house details
  public display(): void {
    console.log('=== House Details ===');
    console.log(`Foundation: ${this.foundation}`);
    console.log(`Structure: ${this.structure}`);
    console.log(`Roof: ${this.roof}`);
    console.log(`Interior: ${this.interior}`);
    console.log(`Garage: ${this.hasGarage ? 'Yes' : 'No'}`);
    console.log(`Garden: ${this.hasGarden ? 'Yes' : 'No'}`);
  }
}
```

```
        console.log(`Swimming Pool: ${this.hasSwimmingPool ? 'Yes' : 'No'}`);
        console.log('=====');
    }
}

// Builder interface - defines the steps to build the product
interface HouseBuilder {
    buildFoundation(): void;
    buildStructure(): void;
    buildRoof(): void;
    buildInterior(): void;
    buildGarage(): void;
    buildGarden(): void;
    buildSwimmingPool(): void;
    getResult(): House;
}

// Concrete Builder 1: Simple House Builder
class SimpleHouseBuilder implements HouseBuilder {
    private house: House = new House();

    buildFoundation(): void {
        this.house.setFoundation('Concrete foundation');
    }

    buildStructure(): void {
        this.house.setStructure('Wood frame structure');
    }

    buildRoof(): void {
        this.house.setRoof('Asphalt shingle roof');
    }

    buildInterior(): void {
        this.house.setInterior('Basic interior with drywall');
    }

    buildGarage(): void {
        this.house.setGarage(false);
    }

    buildGarden(): void {
        this.house.setGarden(false);
    }

    buildSwimmingPool(): void {
        this.house.setSwimmingPool(false);
    }

    getResult(): House {
        return this.house;
    }
}
```

```
// Concrete Builder 2: Luxury House Builder
class LuxuryHouseBuilder implements HouseBuilder {
    private house: House = new House();

    buildFoundation(): void {
        this.house.setFoundation('Reinforced concrete foundation with
basement');
    }

    buildStructure(): void {
        this.house.setStructure('Steel frame structure with brick exterior');
    }

    buildRoof(): void {
        this.house.setRoof('Spanish tile roof with solar panels');
    }

    buildInterior(): void {
        this.house.setInterior('Luxury interior with marble floors and custom
cabinetry');
    }

    buildGarage(): void {
        this.house.setGarage(true);
    }

    buildGarden(): void {
        this.house.setGarden(true);
    }

    buildSwimmingPool(): void {
        this.house.setSwimmingPool(true);
    }

    getResult(): House {
        return this.house;
    }
}

// Director class - orchestrates the construction process
// The director knows the construction algorithm but doesn't know the
details
class ConstructionDirector {
    private builder!: HouseBuilder;

    public setBuilder(builder: HouseBuilder): void {
        this.builder = builder;
    }

    // Construct a basic house
    public buildBasicHouse(): void {
        this.builder.buildFoundation();
        this.builder.buildStructure();
        this.builder.buildRoof();
    }
}
```

```
        this.builder.buildInterior();
    }

    // Construct a full-featured house
    public buildFullHouse(): void {
        this.builder.buildFoundation();
        this.builder.buildStructure();
        this.builder.buildRoof();
        this.builder.buildInterior();
        this.builder.buildGarage();
        this.builder.buildGarden();
        this.builder.buildSwimmingPool();
    }

    // Construct a custom house
    public buildCustomHouse(withGarage: boolean, withGarden: boolean,
withPool: boolean): void {
        this.builder.buildFoundation();
        this.builder.buildStructure();
        this.builder.buildRoof();
        this.builder.buildInterior();
        if (withGarage) this.builder.buildGarage();
        if (withGarden) this.builder.buildGarden();
        if (withPool) this.builder.buildSwimmingPool();
    }
}

// Alternative: Fluent Builder pattern (method chaining)
// This is a common variation where the builder returns itself for chaining
class Computer {
    private cpu: string = '';
    private gpu: string = '';
    private ram: number = 8;
    private storage: number = 256;
    private hasWifi: boolean = false;
    private hasBluetooth: boolean = false;

    public setCpu(cpu: string): void {
        this.cpu = cpu;
    }

    public setGpu(gpu: string): void {
        this.gpu = gpu;
    }

    public setRam(ram: number): void {
        this.ram = ram;
    }

    public setStorage(storage: number): void {
        this.storage = storage;
    }

    public setWifi(wifi: boolean): void {
```

```
        this.hasWifi = wifi;
    }

    public setBluetooth(bluetooth: boolean): void {
        this.hasBluetooth = bluetooth;
    }

    public display(): void {
        console.log('=== Computer Specs ===');
        console.log(`CPU: ${this.cpu}`);
        console.log(`GPU: ${this.gpu}`);
        console.log(`RAM: ${this.ram} GB`);
        console.log(`Storage: ${this.storage} GB`);
        console.log(`WiFi: ${this.hasWifi ? 'Yes' : 'No'}`);
        console.log(`Bluetooth: ${this.hasBluetooth ? 'Yes' : 'No'}`);
        console.log('=====');
    }
}

class ComputerBuilder {
    private computer: Computer = new Computer();

    public setCpu(cpu: string): this {
        this.computer.setCpu(cpu);
        return this;
    }

    public setGpu(gpu: string): this {
        this.computer.setGpu(gpu);
        return this;
    }

    public setRam(ram: number): this {
        this.computer.setRam(ram);
        return this;
    }

    public setStorage(storage: number): this {
        this.computer.setStorage(storage);
        return this;
    }

    public addWifi(): this {
        this.computer.setWifi(true);
        return this;
    }

    public addBluetooth(): this {
        this.computer.setBluetooth(true);
        return this;
    }

    public build(): Computer {
        return this.computer;
    }
}
```

```
}  
}  
  
// Example: SQL Query Builder  
class SQLQueryBuilder {  
  private query: string = '';  
  
  public select(columns: string): this {  
    this.query = `SELECT ${columns}`;  
    return this;  
  }  
  
  public from(table: string): this {  
    this.query += ` FROM ${table}`;  
    return this;  
  }  
  
  public where(condition: string): this {  
    this.query += ` WHERE ${condition}`;  
    return this;  
  }  
  
  public orderBy(column: string, direction: 'ASC' | 'DESC' = 'ASC'): this {  
    this.query += ` ORDER BY ${column} ${direction}`;  
    return this;  
  }  
  
  public limit(count: number): this {  
    this.query += ` LIMIT ${count}`;  
    return this;  
  }  
  
  public build(): string {  
    return this.query + ';';  
  }  
}  
  
// Demo code  
function demoBuilder(): void {  
  console.log('=== Builder Pattern Demo ===\n');  
  
  // Example 1: Classic Builder with Director  
  console.log('--- Classic Builder with Director ---');  
  
  const director = new ConstructionDirector();  
  
  // Build a simple house  
  console.log('\nBuilding Simple House:');  
  const simpleBuilder = new SimpleHouseBuilder();  
  director.setBuilder(simpleBuilder);  
  director.buildBasicHouse();  
  const simpleHouse = simpleBuilder.getResult();  
  simpleHouse.display();  
}
```

```
// Build a luxury house
console.log('\nBuilding Luxury House:');
const luxuryBuilder = new LuxuryHouseBuilder();
director.setBuilder(luxuryBuilder);
director.buildFullHouse();
const luxuryHouse = luxuryBuilder.getResult();
luxuryHouse.display();

// Build a custom house
console.log('\nBuilding Custom House (with garage and garden, no
pool):');
const customBuilder = new SimpleHouseBuilder();
director.setBuilder(customBuilder);
director.buildCustomHouse(true, true, false);
const customHouse = customBuilder.getResult();
customHouse.display();

// Example 2: Fluent Builder (method chaining)
console.log('\n--- Fluent Builder (Method Chaining) ---');

console.log('\nBuilding Gaming Computer:');
const gamingComputer = new ComputerBuilder()
    .setCpu('Intel Core i9-13900K')
    .setGpu('NVIDIA RTX 4090')
    .setRam(32)
    .setStorage(1000)
    .addWifi()
    .addBluetooth()
    .build();
gamingComputer.display();

console.log('\nBuilding Office Computer:');
const officeComputer = new ComputerBuilder()
    .setCpu('Intel Core i5-13400')
    .setGpu('Integrated Graphics')
    .setRam(16)
    .setStorage(512)
    .addWifi()
    .build();
officeComputer.display();

// Example 3: SQL Query Builder
console.log('\n--- SQL Query Builder ---');

const query1 = new SQLQueryBuilder()
    .select('*')
    .from('users')
    .where('age > 18')
    .orderBy('name', 'ASC')
    .limit(10)
    .build();
console.log('Query 1:', query1);

const query2 = new SQLQueryBuilder()
```

```

        .select('name, email')
        .from('customers')
        .where('active = true')
        .orderBy('created_at', 'DESC')
        .build();
    console.log('Query 2:', query2);
}

// Run the demo
demoBuilder();

```

Prototype

Description: Creates new objects by **copying (cloning) an existing object**, rather than creating from scratch.

- **Why it is used:** When object creation is expensive or complex, cloning an existing object is faster and simpler. It avoids the overhead of re-initialization.
- **Where it is commonly used:** Game development (duplicating characters or levels), graphic editors (duplicating shapes), document templates.

Code Example (JAVASCRIPT)

```

/*
 * PROTOTYPE DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Prototype pattern specifies the kinds of objects to create using a
 * prototypical instance, and creates new objects by copying this prototype.
 * It allows you to create new objects by cloning existing ones instead of
 * creating them from scratch.
 *
 * WHY DOES IT EXIST?
 * - When creating an object is expensive or complex
 * - When you want to avoid creating subclasses just to create objects
 * - When you want to hide the concrete classes from the client
 * - When you need to create objects at runtime that are similar to
 * existing objects
 *
 * WHEN TO USE IT?
 * - When the classes to instantiate are specified at runtime
 * - When you want to avoid building a class hierarchy of factories that
 * parallels the class hierarchy of products
 * - When instances of a class can have one of only a few different
 * combinations of state
 * - When creating an object is expensive (e.g., requires database access,
 * network I/O, or complex initialization)
 * - Common use cases: game objects, document templates, graphic shapes,
 * configuration objects
 *
 * PROS:

```

```

* - Allows you to clone objects without coupling to their concrete classes
* - Can avoid repetitive initialization code
* - Can be more efficient than creating objects from scratch
* - Lets you add or remove products at runtime
* - Works well with complex object graphs
*
* CONS:
* - Cloning complex objects with circular references can be tricky
* - Shallow vs. deep copying must be carefully considered
* - May be difficult to implement if the object's internal state is not
easily accessible
* - Can violate encapsulation if the clone method needs access to private
fields
*
* REAL-WORLD ANALOGY:
* Think of a cell division in biology. A cell (prototype) divides to
create an identical copy of itself. The new cell starts with the same DNA
and characteristics as the original, but can then evolve independently.
Similarly, in software, you can clone an object and then modify it as
needed.
*/

// Prototype interface - declares the clone method
interface Prototype {
    clone(): Prototype;
    display(): void;
    getType(): string;
}

// Concrete Prototype 1: Rectangle
class Rectangle implements Prototype {
    constructor(
        private width: number,
        private height: number,
        private color: string
    ) {}

    // Clone method using Object.assign for shallow copy
    clone(): Prototype {
        return new Rectangle(this.width, this.height, this.color);
    }

    display(): void {
        console.log(`Rectangle: ${this.width}x${this.height}, Color:
${this.color}`);
    }

    getType(): string {
        return 'Rectangle';
    }

    // Methods to modify the cloned object
    public setWidth(width: number): void {
        this.width = width;

```

```
}

public setHeight(height: number): void {
    this.height = height;
}

public setColor(color: string): void {
    this.color = color;
}
}

// Concrete Prototype 2: Circle
class Circle implements Prototype {
    constructor(
        private radius: number,
        private color: string
    ) {}

    clone(): Prototype {
        return new Circle(this.radius, this.color);
    }

    display(): void {
        console.log(`Circle: Radius ${this.radius}, Color: ${this.color}`);
    }

    getType(): string {
        return 'Circle';
    }

    public setRadius(radius: number): void {
        this.radius = radius;
    }

    public setColor(color: string): void {
        this.color = color;
    }
}

// Prototype Registry - manages a collection of prototypes
class PrototypeRegistry {
    private prototypes: Map<string, Prototype> = new Map();

    public registerPrototype(key: string, prototype: Prototype): void {
        this.prototypes.set(key, prototype);
    }

    public clone(key: string): Prototype | null {
        const prototype = this.prototypes.get(key);
        return prototype ? prototype.clone() : null;
    }
}

// Example: Complex object with deep copy
```

```
class Employee implements Prototype {
  constructor(
    private name: string,
    private department: string,
    private salary: number,
    private skills: string[] = []
  ) {}

  // Deep copy constructor
  clone(): Prototype {
    // Create a deep copy of the skills array
    const skillsCopy = [...this.skills];
    return new Employee(this.name, this.department, this.salary,
skillsCopy);
  }

  display(): void {
    console.log(`Employee: ${this.name}, Dept: ${this.department}, Salary:
${this.salary}`);
    console.log(`Skills: ${this.skills.join(', ')} `);
  }

  getType(): string {
    return 'Employee';
  }

  public addSkill(skill: string): void {
    this.skills.push(skill);
  }

  public setName(name: string): void {
    this.name = name;
  }

  public setSalary(salary: number): void {
    this.salary = salary;
  }
}

// Example: Document with shallow vs deep copy
class Document implements Prototype {
  constructor(
    private title: string,
    private content: string
  ) {}

  clone(): Prototype {
    // For strings, this is effectively a deep copy since strings are
immutable in TypeScript
    return new Document(this.title, this.content);
  }

  display(): void {
    console.log(`Document: ${this.title}`);
  }
}
```

```
        console.log(`Content: ${this.content}`);
    }

    getType(): string {
        return 'Document';
    }

    public appendContent(text: string): void {
        this.content += text;
    }

    public setTitle(title: string): void {
        this.title = title;
    }
}

// Example: Game Character with prototype
class GameCharacter implements Prototype {
    constructor(
        private name: string,
        private health: number,
        private attack: number,
        private defense: number,
        private inventory: string[] = []
    ) {}

    clone(): Prototype {
        return new GameCharacter(
            this.name,
            this.health,
            this.attack,
            this.defense,
            [...this.inventory]
        );
    }

    display(): void {
        console.log(`Character: ${this.name}`);
        console.log(`  Health: ${this.health}, Attack: ${this.attack}, Defense: ${this.defense}`);
        console.log(`  Inventory: ${this.inventory.join(', ')} `);
    }

    getType(): string {
        return 'GameCharacter';
    }

    public addItem(item: string): void {
        this.inventory.push(item);
    }

    public setName(name: string): void {
        this.name = name;
    }
}
```

```
    public takeDamage(amount: number): void {
        this.health = Math.max(0, this.health - amount);
    }
}

// Demo code
function demoPrototype(): void {
    console.log('=== Prototype Pattern Demo ===\n');

    // Example 1: Basic Prototype with Registry
    console.log('--- Basic Prototype with Registry ---');

    const registry = new PrototypeRegistry();

    // Register prototypes
    registry.registerPrototype('red_rectangle', new Rectangle(10, 5, 'red'));
    registry.registerPrototype('blue_circle', new Circle(7, 'blue'));

    // Clone and modify prototypes
    console.log('\nCloning red rectangle:');
    const rect1 = registry.clone('red_rectangle');
    if (rect1) rect1.display();

    console.log('\nCloning and modifying red rectangle:');
    const rect2 = registry.clone('red_rectangle');
    if (rect2) {
        const rect2Typed = rect2 as Rectangle;
        rect2Typed.setWidth(20);
        rect2Typed.setColor('green');
        rect2Typed.display();
    }

    console.log('\nCloning blue circle:');
    const circle1 = registry.clone('blue_circle');
    if (circle1) circle1.display();

    // Example 2: Complex object with deep copy
    console.log('\n--- Complex Object with Deep Copy ---');

    const originalEmployee = new Employee('John Doe', 'Engineering', 75000);
    originalEmployee.addSkill('TypeScript');
    originalEmployee.addSkill('Python');
    originalEmployee.addSkill('JavaScript');

    console.log('\nOriginal employee:');
    originalEmployee.display();

    console.log('\nCloned employee:');
    const clonedEmployee = originalEmployee.clone() as Employee;
    clonedEmployee.setName('Jane Smith');
    clonedEmployee.setSalary(80000);
    clonedEmployee.addSkill('Go');
    clonedEmployee.display();
}
```

```
console.log('\nOriginal employee (unchanged):');
originalEmployee.display();

// Example 3: Document with deep copy
console.log('\n--- Document with Deep Copy ---');

const originalDoc = new Document('Project Plan', 'This document outlines
the project plan.');
```

originalDoc.display();

```
console.log('\nCloned document:');
const clonedDoc = originalDoc.clone() as Document;
clonedDoc.setTitle('Project Plan - Copy');
clonedDoc.appendContent(' It includes milestones and deliverables.');
```

clonedDoc.display();

```
console.log('\nOriginal document (unchanged):');
originalDoc.display();

// Example 4: Game Character with prototype
console.log('\n--- Game Character with Prototype ---');
```

```
const warriorTemplate = new GameCharacter('Warrior', 100, 25, 15);
warriorTemplate.addItem('Sword');
warriorTemplate.addItem('Shield');
```

```
console.log('\nCreating warrior from template:');
const warrior1 = warriorTemplate.clone() as GameCharacter;
warrior1.setName('Arthur');
```

warrior1.display();

```
console.log('\nCreating another warrior from template:');
const warrior2 = warriorTemplate.clone() as GameCharacter;
warrior2.setName('Lancelot');
```

warrior2.takeDamage(20);

```
warrior2.addItem('Potion');
warrior2.display();

console.log('\nTemplate (unchanged):');
warriorTemplate.display();
}
```

// Run the demo

```
demoPrototype();
```

2. Structural Design Patterns

Overview

Structural patterns deal with **how classes and objects are composed** to form larger structures. They simplify the structure by identifying relationships between entities and help ensure that when one part of a system changes, the entire structure does not need to change.

Adapter

Description: Acts as a **bridge between two incompatible interfaces**. It wraps an existing class with a new interface so that it becomes compatible with the client's expectations.

- **Why it is used:** To allow classes with incompatible interfaces to work together without modifying their source code.
 - **Where it is commonly used:** Integrating third-party libraries, legacy code integration, data format conversion (e.g., XML to JSON).
-

Code Example (JAVASCRIPT)

```
/*
 * ADAPTER DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Adapter pattern allows objects with incompatible interfaces to
collaborate. It acts as a bridge between two incompatible interfaces.
 *
 * WHY DOES IT EXIST?
 * - When you want to use an existing class whose interface doesn't match
the one you need
 * - When you want to create a reusable class that cooperates with
unrelated classes with incompatible interfaces
 * - When you need to use several existing subclasses, but adapting each to
match your interface would be impractical
 *
 * WHEN TO USE IT?
 * - When you want to use a third-party class but its interface doesn't
match your system's interface
 * - When you need to integrate new components into an existing system
without modifying existing code
 * - When you want to reuse existing classes that have similar
functionality but different interfaces
 * - Common use cases: integrating third-party libraries, legacy code
integration, data format conversion
 *
 * PROS:
 * - Single Responsibility Principle (separates conversion logic from
business logic)
 * - Open/Closed Principle (introduce new adapters without modifying
existing code)
 * - Allows incompatible classes to work together
 * - Improves code reusability
 *
 * CONS:
```

```
* - Increases overall code complexity
* - Can introduce additional layers of indirection
* - May not be suitable for simple cases where direct modification is
easier
* - Can make the code harder to understand due to multiple layers
*
* REAL-WORLD ANALOGY:
* Think of a power adapter when traveling abroad. Your device (client)
expects a certain plug shape (target interface), but the wall outlet
(adaptee) has a different shape. The adapter converts between the two,
allowing your device to work with the foreign outlet without modifying
either the device or the wall.
*/

// Target Interface - the interface that the client expects
interface MediaPlayer {
  play(audioType: string, fileName: string): void;
}

// Adaptee - the class with an incompatible interface
class AdvancedMediaPlayer {
  playVlc(fileName: string): void {
    console.log(`Playing vlc file: ${fileName}`);
  }

  playMp4(fileName: string): void {
    console.log(`Playing mp4 file: ${fileName}`);
  }
}

// Adapter - makes the Adaptee's interface compatible with the Target
interface
class MediaAdapter implements MediaPlayer {
  private advancedMusicPlayer: AdvancedMediaPlayer;

  constructor() {
    this.advancedMusicPlayer = new AdvancedMediaPlayer();
  }

  play(audioType: string, fileName: string): void {
    if (audioType === 'vlc') {
      this.advancedMusicPlayer.playVlc(fileName);
    } else if (audioType === 'mp4') {
      this.advancedMusicPlayer.playMp4(fileName);
    } else {
      console.log(`Invalid media. ${audioType} format not supported`);
    }
  }
}

// Client - uses the Target interface
class AudioPlayer implements MediaPlayer {
  private mediaAdapter?: MediaAdapter;
```

```
play(audioType: string, fileName: string): void {
  // Built-in support for mp3
  if (audioType === 'mp3') {
    console.log(`Playing mp3 file: ${fileName}`);
  }
  // MediaAdapter provides support for other formats
  else if (audioType === 'vlc' || audioType === 'mp4') {
    this.mediaAdapter = new MediaAdapter();
    this.mediaAdapter.play(audioType, fileName);
  } else {
    console.log(`Invalid media. ${audioType} format not supported`);
  }
}

// Example 2: Payment Processing Adapter
// This example shows a different scenario: payment processing

// Target Interface
interface PaymentProcessor {
  processPayment(amount: number): void;
}

// Adaptee - incompatible payment system
class StripePaymentSystem {
  makeStripePayment(amount: number): void {
    console.log(`Processing ${amount} via Stripe`);
  }
}

// Adaptee - another incompatible payment system
class PayPalPaymentSystem {
  sendPayPalRequest(amount: number): void {
    console.log(`Processing ${amount} via PayPal`);
  }
}

// Adapter for Stripe
class StripeAdapter implements PaymentProcessor {
  private stripeSystem: StripePaymentSystem;

  constructor() {
    this.stripeSystem = new StripePaymentSystem();
  }

  processPayment(amount: number): void {
    this.stripeSystem.makeStripePayment(amount);
  }
}

// Adapter for PayPal
class PayPalAdapter implements PaymentProcessor {
  private paypalSystem: PayPalPaymentSystem;
```

```
    constructor() {
        this.payPalSystem = new PayPalPaymentSystem();
    }

    processPayment(amount: number): void {
        this.payPalSystem.sendPayPalRequest(amount);
    }
}

// Example 3: Two-way Adapter
// Allows both interfaces to work with each other

// Interface A
interface CelsiusTemperature {
    getTemperatureInCelsius(): number;
    setTemperatureInCelsius(temp: number): void;
}

// Interface B
interface FahrenheitTemperature {
    getTemperatureInFahrenheit(): number;
    setTemperatureInFahrenheit(temp: number): void;
}

// Two-way adapter
class TemperatureAdapter implements CelsiusTemperature,
FahrenheitTemperature {
    private temperatureInCelsius: number;

    constructor(temp: number = 0) {
        this.temperatureInCelsius = temp;
    }

    // Celsius interface implementation
    getTemperatureInCelsius(): number {
        return this.temperatureInCelsius;
    }

    setTemperatureInCelsius(temp: number): void {
        this.temperatureInCelsius = temp;
    }

    // Fahrenheit interface implementation
    getTemperatureInFahrenheit(): number {
        return (this.temperatureInCelsius * 9 / 5) + 32;
    }

    setTemperatureInFahrenheit(temp: number): void {
        this.temperatureInCelsius = (temp - 32) * 5 / 9;
    }
}

// Demo code
function demoAdapter(): void {
```

```
console.log('=== Adapter Pattern Demo ===\n');

// Example 1: Media Player Adapter
console.log('--- Media Player Adapter ---');

const audioPlayer = new AudioPlayer();

audioPlayer.play('mp3', 'beyond the horizon.mp3');
audioPlayer.play('mp4', 'alone again.mp4');
audioPlayer.play('vlc', 'far far away.vlc');
audioPlayer.play('avi', 'mind me.avi');

console.log();

// Example 2: Payment Processing Adapter
console.log('--- Payment Processing Adapter ---');

const stripeAdapter: PaymentProcessor = new StripeAdapter();
stripeAdapter.processPayment(100.50);

const paypalAdapter: PaymentProcessor = new PayPalAdapter();
paypalAdapter.processPayment(75.25);

console.log();

// Example 3: Two-way Temperature Adapter
console.log('--- Two-way Temperature Adapter ---');

const tempAdapter = new TemperatureAdapter();

// Set using Celsius interface
tempAdapter.setTemperatureInCelsius(25.0);
console.log(`Temperature in Celsius:
${tempAdapter.getTemperatureInCelsius()}°C`);
console.log(`Temperature in Fahrenheit:
${tempAdapter.getTemperatureInFahrenheit()}°F`);

// Set using Fahrenheit interface
tempAdapter.setTemperatureInFahrenheit(77.0);
console.log(`\nTemperature in Celsius:
${tempAdapter.getTemperatureInCelsius()}°C`);
console.log(`Temperature in Fahrenheit:
${tempAdapter.getTemperatureInFahrenheit()}°F`);
}

// Run the demo
demoAdapter();
```

Bridge

Description: Decouples an abstraction from its implementation so that the two can vary independently.

- **Why it is used:** To avoid a permanent binding between an abstraction and its implementation and to allow both to be extended independently.
 - **Where it is commonly used:** Device driver systems, rendering engines (OpenGL vs. DirectX), platform-independent UI frameworks.
-

Code Example (JAVASCRIPT)

```
/*
 * BRIDGE DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Bridge pattern separates abstraction from implementation so that
both can vary independently.
 * It uses composition instead of inheritance to decouple the abstraction
from its implementation.
 *
 * WHY DOES IT EXIST?
 * - When you want to avoid a permanent binding between an abstraction and
its implementation
 * - When both the abstraction and the implementation should be extensible
by subclassing
 * - When changes in the implementation should not affect clients
 * - When you have a class hierarchy that explodes due to combining
multiple dimensions
 *
 * WHEN TO USE IT?
 * - When you want to split a monolithic class that has several variants of
some functionality
 * - When you need to extend a class in several independent dimensions
 * - When you want to be able to switch implementations at runtime
 * - Common use cases: UI frameworks (different controls on different
platforms), shape rendering, device drivers
 *
 * PROS:
 * - Separates abstraction from implementation
 * - Improves extensibility (can extend abstractions and implementations
independently)
 * - Hides implementation details from clients
 * - Reduces the number of subclasses needed (avoids combinatorial
explosion)
 *
 * CONS:
 * - Increases code complexity
 * - May require additional indirection
 * - Can make the code harder to understand initially
 * - Might be overkill for simple scenarios
 *
 * REAL-WORLD ANALOGY:
 * Think of a TV remote control (abstraction) and the TV itself
(implementation). The remote control doesn't need to know the internal
workings of the TV. You can use the same remote with different TV brands,
```

```
and different remotes with the same TV. The remote (abstraction) is bridged
to the TV (implementation) through a standard interface (infrared signals).
*/

// Implementation Interface - defines the operations for all concrete
implementations
interface DrawingAPI {
  drawCircle(x: number, y: number, radius: number): void;
  drawRectangle(x: number, y: number, width: number, height: number): void;
}

// Concrete Implementation 1: Red Circle API
class RedCircleAPI implements DrawingAPI {
  drawCircle(x: number, y: number, radius: number): void {
    console.log(`Drawing Circle [color: red, center: (${x}, ${y}), radius:
${radius}]`);
  }

  drawRectangle(x: number, y: number, width: number, height: number): void
{
    console.log(`Drawing Rectangle [color: red, top-left: (${x}, ${y}),
width: ${width}, height: ${height}]`);
  }
}

// Concrete Implementation 2: Green Circle API
class GreenCircleAPI implements DrawingAPI {
  drawCircle(x: number, y: number, radius: number): void {
    console.log(`Drawing Circle [color: green, center: (${x}, ${y}),
radius: ${radius}]`);
  }

  drawRectangle(x: number, y: number, width: number, height: number): void
{
    console.log(`Drawing Rectangle [color: green, top-left: (${x}, ${y}),
width: ${width}, height: ${height}]`);
  }
}

// Abstraction - defines the abstraction's interface and maintains a
reference to an implementation
abstract class Shape {
  protected drawingAPI: DrawingAPI;

  constructor(api: DrawingAPI) {
    this.drawingAPI = api;
  }

  abstract draw(): void;
  abstract resize(percentage: number): void;
}

// Refined Abstraction 1: Circle
class Circle extends Shape {
```

```
private x: number;
private y: number;
private radius: number;

constructor(x: number, y: number, radius: number, api: DrawingAPI) {
  super(api);
  this.x = x;
  this.y = y;
  this.radius = radius;
}

draw(): void {
  this.drawingAPI.drawCircle(this.x, this.y, this.radius);
}

resize(percentage: number): void {
  this.radius *= percentage / 100;
  console.log(`Resizing circle to ${this.radius} radius`);
}
}

// Refined Abstraction 2: Rectangle
class Rectangle extends Shape {
  private x: number;
  private y: number;
  private width: number;
  private height: number;

  constructor(x: number, y: number, width: number, height: number, api:
DrawingAPI) {
    super(api);
    this.x = x;
    this.y = y;
    this.width = width;
    this.height = height;
  }

  draw(): void {
    this.drawingAPI.drawRectangle(this.x, this.y, this.width, this.height);
  }

  resize(percentage: number): void {
    this.width *= percentage / 100;
    this.height *= percentage / 100;
    console.log(`Resizing rectangle to ${this.width}x${this.height}`);
  }
}

// Example 2: Device and Remote Control Bridge
// This shows how the Bridge pattern can be used for device control

// Implementation Interface
interface Device {
  turnOn(): void;
}
```

```
    turnOff(): void;
    setVolume(volume: number): void;
    getVolume(): number;
}

// Concrete Implementation 1: TV
class Television implements Device {
    private on: boolean = false;
    private volume: number = 10;

    turnOn(): void {
        this.on = true;
        console.log('TV is now ON');
    }

    turnOff(): void {
        this.on = false;
        console.log('TV is now OFF');
    }

    setVolume(volume: number): void {
        if (this.on) {
            this.volume = volume;
            console.log(`TV volume set to ${volume}`);
        } else {
            console.log('Cannot set volume: TV is OFF');
        }
    }

    getVolume(): number {
        return this.volume;
    }
}

// Concrete Implementation 2: Radio
class Radio implements Device {
    private on: boolean = false;
    private volume: number = 5;

    turnOn(): void {
        this.on = true;
        console.log('Radio is now ON');
    }

    turnOff(): void {
        this.on = false;
        console.log('Radio is now OFF');
    }

    setVolume(volume: number): void {
        if (this.on) {
            this.volume = volume;
            console.log(`Radio volume set to ${volume}`);
        } else {

```

```
        console.log('Cannot set volume: Radio is OFF');
    }
}

getVolume(): number {
    return this.volume;
}
}

// Abstraction: Remote Control
class RemoteControl {
    protected device: Device;

    constructor(device: Device) {
        this.device = device;
    }

    togglePower(): void {
        if (this.device.getVolume() >= 0) {
            this.device.turnOff();
        } else {
            this.device.turnOn();
        }
    }

    volumeUp(): void {
        const currentVolume = this.device.getVolume();
        this.device.setVolume(currentVolume + 1);
    }

    volumeDown(): void {
        const currentVolume = this.device.getVolume();
        this.device.setVolume(currentVolume - 1);
    }
}

// Refined Abstraction: Advanced Remote Control
class AdvancedRemoteControl extends RemoteControl {
    mute(): void {
        console.log('Muting device');
        this.device.setVolume(0);
    }
}

// Demo code
function demoBridge(): void {
    console.log('=== Bridge Pattern Demo ===\n');

    // Example 1: Shape Drawing Bridge
    console.log('--- Shape Drawing Bridge ---');

    const redCircle = new Circle(100, 100, 10, new RedCircleAPI());
    const greenCircle = new Circle(100, 100, 10, new GreenCircleAPI());
```

```
redCircle.draw();
greenCircle.draw();

const redRectangle = new Rectangle(50, 50, 20, 30, new RedCircleAPI());
const greenRectangle = new Rectangle(50, 50, 20, 30, new
GreenCircleAPI());

redRectangle.draw();
greenRectangle.draw();

console.log('\nResizing shapes:');
redCircle.resize(150);
redCircle.draw();

console.log();

// Example 2: Device and Remote Control Bridge
console.log('--- Device and Remote Control Bridge ---');

const tv = new Television();
const tvRemote = new RemoteControl(tv);

console.log('\nControlling TV with basic remote:');
tvRemote.togglePower();
tvRemote.volumeUp();
tvRemote.volumeUp();
tvRemote.volumeDown();
tvRemote.togglePower();

const radio = new Radio();
const advancedRemote = new AdvancedRemoteControl(radio);

console.log('\nControlling Radio with advanced remote:');
advancedRemote.togglePower();
advancedRemote.volumeUp();
advancedRemote.mute();
advancedRemote.togglePower();
}

// Run the demo
demoBridge();
```

Composite

Description: Composes objects into **tree structures to represent part-whole hierarchies**. It lets clients treat individual objects and compositions of objects uniformly.

- **Why it is used:** To simplify client code that deals with tree structures by treating individual items and groups in the same way.
- **Where it is commonly used:** File system structures (files and folders), UI component trees, organization hierarchies.

Code Example (JAVASCRIPT)

```
/*
 * COMPOSITE DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Composite pattern lets you compose objects into tree structures and
work with these structures as if they were individual objects.
 * It treats individual objects and compositions of objects uniformly.
 *
 * WHY DOES IT EXIST?
 * - When you want to represent part-whole hierarchies of objects
 * - When you want clients to ignore the difference between compositions of
objects and individual objects
 * - When you want to treat individual objects and compositions uniformly
 *
 * WHEN TO USE IT?
 * - When you have a tree structure of objects (e.g., file systems,
organization charts, UI component hierarchies)
 * - When you want to simplify client code by treating individual and
composite objects the same way
 * - When the structure of your objects forms a recursive hierarchy
 * - Common use cases: file systems, graphics scenes, UI component trees,
organization structures
 *
 * PROS:
 * - Simplifies client code (can treat complex and simple objects
uniformly)
 * - Open/Closed Principle (easy to add new kinds of components)
 * - Flexible tree structure (can build complex structures from simple
parts)
 * - Makes it easier to add new types of components
 *
 * CONS:
 * - Can make the design overly general (hard to restrict components)
 * - May be difficult to restrict the types of components in a composite
 * - Can make the code harder to understand due to recursion
 * - Performance overhead for traversing large trees
 *
 * REAL-WORLD ANALOGY:
 * Think of a file system. A file is a simple object (leaf), while a folder
is a composite that can contain both files and other folders. When you
delete a folder, you want to delete everything inside it recursively. The
file system treats files and folders uniformly in many operations (like
copying, moving, or calculating size).
 */

// Component Interface - declares the interface for objects in the
composition
abstract class FileSystemComponent {
  abstract display(indent: string): void;
  abstract getSize(): number;
```

```
add(component: FileSystemComponent): void {
    console.log('Cannot add to a file');
}

remove(component: FileSystemComponent): void {
    console.log('Cannot remove from a file');
}

getChild(index: number): FileSystemComponent | null {
    return null;
}
}

// Leaf - represents individual objects in the composition
class File extends FileSystemComponent {
    private name: string;
    private size: number;

    constructor(name: string, size: number) {
        super();
        this.name = name;
        this.size = size;
    }

    display(indent: string = ''): void {
        console.log(`${indent}File: ${this.name} (${this.size} bytes)`);
    }

    getSize(): number {
        return this.size;
    }
}

// Composite - represents groups of objects that can contain other
// composites
class Directory extends FileSystemComponent {
    private name: string;
    private children: FileSystemComponent[] = [];

    constructor(name: string) {
        super();
        this.name = name;
    }

    add(component: FileSystemComponent): void {
        this.children.push(component);
    }

    remove(component: FileSystemComponent): void {
        const index = this.children.indexOf(component);
        if (index > -1) {
            this.children.splice(index, 1);
        }
    }
}
```

```

    }

    getChild(index: number): FileSystemComponent | null {
        if (index >= 0 && index < this.children.length) {
            return this.children[index];
        }
        return null;
    }

    display(indent: string = ''): void {
        console.log(`${indent}Directory: ${this.name} (${this.getSize()}
bytes)`);
        for (const child of this.children) {
            child.display(indent + ' ');
        }
    }

    getSize(): number {
        let totalSize = 0;
        for (const child of this.children) {
            totalSize += child.getSize();
        }
        return totalSize;
    }
}

// Example 2: Organization Structure
// Shows how Composite can be used for organizational hierarchies

abstract class Employee {
    abstract print(indent: string): void;

    add(employee: Employee): void {
        console.log('Cannot add to a regular employee');
    }

    remove(employee: Employee): void {
        console.log('Cannot remove from a regular employee');
    }
}

class Worker extends Employee {
    private name: string;
    private position: string;

    constructor(name: string, position: string) {
        super();
        this.name = name;
        this.position = position;
    }

    print(indent: string = ''): void {
        console.log(`${indent}Employee: ${this.name} - ${this.position}`);
    }
}

```

```
}

class Manager extends Employee {
  private name: string;
  private title: string;
  private subordinates: Employee[] = [];

  constructor(name: string, title: string) {
    super();
    this.name = name;
    this.title = title;
  }

  add(employee: Employee): void {
    this.subordinates.push(employee);
  }

  remove(employee: Employee): void {
    const index = this.subordinates.indexOf(employee);
    if (index > -1) {
      this.subordinates.splice(index, 1);
    }
  }

  print(indent: string = ''): void {
    console.log(`${indent}Manager: ${this.name} - ${this.title}`);
    for (const subordinate of this.subordinates) {
      subordinate.print(indent + ' ');
    }
  }
}

// Example 3: Graphic Shapes Composite
// Shows how Composite can be used for graphics

abstract class Graphic {
  abstract draw(indent: string): void;
}

class Circle extends Graphic {
  private name: string;

  constructor(name: string) {
    super();
    this.name = name;
  }

  draw(indent: string = ''): void {
    console.log(`${indent}Drawing Circle: ${this.name}`);
  }
}

class Square extends Graphic {
  private name: string;
```

```
    constructor(name: string) {
        super();
        this.name = name;
    }

    draw(indent: string = ''): void {
        console.log(`${indent}Drawing Square: ${this.name}`);
    }
}

class GraphicGroup extends Graphic {
    private name: string;
    private graphics: Graphic[] = [];

    constructor(name: string) {
        super();
        this.name = name;
    }

    add(graphic: Graphic): void {
        this.graphics.push(graphic);
    }

    remove(graphic: Graphic): void {
        const index = this.graphics.indexOf(graphic);
        if (index > -1) {
            this.graphics.splice(index, 1);
        }
    }

    draw(indent: string = ''): void {
        console.log(`${indent}Group: ${this.name}`);
        for (const graphic of this.graphics) {
            graphic.draw(indent + ' ');
        }
    }
}

// Demo code
function demoComposite(): void {
    console.log('=== Composite Pattern Demo ===\n');

    // Example 1: File System
    console.log('--- File System Example ---');

    const file1 = new File('file1.txt', 100);
    const file2 = new File('file2.txt', 200);
    const file3 = new File('file3.txt', 150);

    const dir1 = new Directory('Documents');
    const dir2 = new Directory('Pictures');
    const rootDir = new Directory('Root');
```

```
dir1.add(file1);
dir1.add(file2);

dir2.add(file3);

rootDir.add(dir1);
rootDir.add(dir2);

console.log('\nFile System Structure:');
rootDir.display();

console.log(`\nTotal size of Root directory: ${rootDir.getSize()}
bytes`);

console.log();

// Example 2: Organization Structure
console.log('--- Organization Structure Example ---');

const ceo = new Manager('John Smith', 'CEO');
const cto = new Manager('Alice Johnson', 'CTO');
const cfo = new Manager('Bob Williams', 'CFO');

const dev1 = new Worker('Developer 1', 'Senior Developer');
const dev2 = new Worker('Developer 2', 'Junior Developer');
const accountant = new Worker('Accountant', 'Senior Accountant');

cto.add(dev1);
cto.add(dev2);
cfo.add(accountant);

ceo.add(cto);
ceo.add(cfo);

console.log('\nOrganization Structure:');
ceo.print();

console.log();

// Example 3: Graphic Shapes
console.log('--- Graphic Shapes Example ---');

const circle1 = new Circle('Red Circle');
const circle2 = new Circle('Blue Circle');
const square1 = new Square('Green Square');

const group1 = new GraphicGroup('Shapes Group 1');
const group2 = new GraphicGroup('Shapes Group 2');
const mainGroup = new GraphicGroup('Main Group');

group1.add(circle1);
group1.add(square1);

group2.add(circle2);
```

```
mainGroup.add(group1);
mainGroup.add(group2);

console.log('\nGraphic Structure:');
mainGroup.draw();
}

// Run the demo
demoComposite();
```

Decorator

Description: Adds new behavior or responsibilities to an object dynamically, without altering its class. It wraps the original object and adds functionality on top.

- **Why it is used:** To extend object functionality at runtime without using inheritance, keeping classes flexible and following the Open/Closed Principle.
- **Where it is commonly used:** I/O streams in Java, middleware in web frameworks, text formatting systems.

Code Example (JAVASCRIPT)

```
/*
 * DECORATOR DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Decorator pattern lets you attach new behaviors to objects by
 * placing these objects inside special wrapper objects that contain the
 * behaviors.
 * It provides a flexible alternative to inheritance for extending
 * functionality.
 *
 * WHY DOES IT EXIST?
 * - When you want to add responsibilities to individual objects
 *   dynamically without affecting other objects
 * - When extending a class by subclassing is impractical (too many
 *   subclasses)
 * - When you want to add and remove responsibilities at runtime
 *
 * WHEN TO USE IT?
 * - When you need to add responsibilities to individual objects without
 *   affecting other objects of the same class
 * - When the class definition may be hidden or otherwise unavailable for
 *   subclassing
 * - When you want to add functionality to objects in a transparent way
 *   (clients shouldn't know the difference)
 * - Common use cases: UI components (scrollbars, borders), data streams
 *   (compression, encryption), logging, caching
 */
```

```
* PROS:
* - Single Responsibility Principle (divides functionality into small
classes)
* - Open/Closed Principle (can extend behavior without modifying existing
code)
* - Flexible alternative to subclassing
* - Can add/remove responsibilities at runtime
*
* CONS:
* - Can result in many small classes (code complexity)
* - Can make it hard to remove wrapped decorators
* - Can make debugging difficult due to many layers
* - Initial setup can be complex
*
* REAL-WORLD ANALOGY:
* Think of ordering coffee. You start with a basic coffee (component),
then you can add decorators like milk, sugar, whipped cream, or syrup. Each
decorator adds its own behavior (flavor, cost) while still being treated as
coffee. You can combine multiple decorators in any order.
*/

// Component Interface - defines the interface for objects that can have
responsibilities added
interface Coffee {
  getCost(): number;
  getDescription(): string;
}

// Concrete Component - the basic object that can have responsibilities
added
class SimpleCoffee implements Coffee {
  getCost(): number {
    return 2.0;
  }

  getDescription(): string {
    return 'Simple Coffee';
  }
}

// Decorator - implements the Component interface and wraps a Component
abstract class CoffeeDecorator implements Coffee {
  protected decoratedCoffee: Coffee;

  constructor(coffee: Coffee) {
    this.decoratedCoffee = coffee;
  }

  getCost(): number {
    return this.decoratedCoffee.getCost();
  }

  getDescription(): string {
    return this.decoratedCoffee.getDescription();
  }
}
```

```
}  
}  
  
// Concrete Decorator 1: Milk  
class MilkDecorator extends CoffeeDecorator {  
  getCost(): number {  
    return super.getCost() + 0.5;  
  }  
  
  getDescription(): string {  
    return super.getDescription() + ', Milk';  
  }  
}  
  
// Concrete Decorator 2: Sugar  
class SugarDecorator extends CoffeeDecorator {  
  getCost(): number {  
    return super.getCost() + 0.2;  
  }  
  
  getDescription(): string {  
    return super.getDescription() + ', Sugar';  
  }  
}  
  
// Concrete Decorator 3: Whipped Cream  
class WhippedCreamDecorator extends CoffeeDecorator {  
  getCost(): number {  
    return super.getCost() + 1.0;  
  }  
  
  getDescription(): string {  
    return super.getDescription() + ', Whipped Cream';  
  }  
}  
  
// Example 2: Text Formatting Decorator  
// Shows how decorators can be used for text processing  
  
interface TextComponent {  
  getText(): string;  
}  
  
class SimpleText implements TextComponent {  
  constructor(private text: string) {}  
  
  getText(): string {  
    return this.text;  
  }  
}  
  
abstract class TextDecorator implements TextComponent {  
  protected wrappedText: TextComponent;
```

```
    constructor(text: TextComponent) {
        this.wrappedText = text;
    }

    getText(): string {
        return this.wrappedText.getText();
    }
}

class BoldDecorator extends TextDecorator {
    getText(): string {
        return `${super.getText()}</b>`;
    }
}

class ItalicDecorator extends TextDecorator {
    getText(): string {
        return `${super.getText()}</i>`;
    }
}

class UnderlineDecorator extends TextDecorator {
    getText(): string {
        return `${super.getText()}</u>`;
    }
}

// Example 3: Data Stream Decorator
// Shows how decorators can be used for data processing

interface DataSource {
    writeData(data: string): void;
    readData(): string;
}

class FileDataSource implements DataSource {
    private data: string = '';

    writeData(data: string): void {
        this.data = data;
        console.log(`Writing data to file: ${data}`);
    }

    readData(): string {
        console.log(`Reading data from file: ${this.data}`);
        return this.data;
    }
}

abstract class DataSourceDecorator implements DataSource {
    protected wrappedSource: DataSource;

    constructor(source: DataSource) {
        this.wrappedSource = source;
    }
}
```

```
}

writeData(data: string): void {
  this.wrappedSource.writeData(data);
}

readData(): string {
  return this.wrappedSource.readData();
}
}

class EncryptionDecorator extends DataSourceDecorator {
  writeData(data: string): void {
    const encrypted = `[ENCRYPTED]${data}[ENCRYPTED]`;
    console.log('Encrypting data...');
    this.wrappedSource.writeData(encrypted);
  }

  readData(): string {
    const data = this.wrappedSource.readData();
    console.log('Decrypting data...');
    // Remove encryption markers for demo
    if (data.startsWith('[ENCRYPTED]')) {
      return data.substring(11, data.length - 11);
    }
    return data;
  }
}

class CompressionDecorator extends DataSourceDecorator {
  writeData(data: string): void {
    const compressed = `[COMPRESSED]${data}[COMPRESSED]`;
    console.log('Compressing data...');
    this.wrappedSource.writeData(compressed);
  }

  readData(): string {
    const data = this.wrappedSource.readData();
    console.log('Decompressing data...');
    // Remove compression markers for demo
    if (data.startsWith('[COMPRESSED]')) {
      return data.substring(12, data.length - 12);
    }
    return data;
  }
}

// Demo code
function demoDecorator(): void {
  console.log('=== Decorator Pattern Demo ===\n');

  // Example 1: Coffee Shop
  console.log('--- Coffee Shop Example ---');
```

```
let coffee: Coffee = new SimpleCoffee();
console.log(`${coffee.getDescription()} ${coffee.getCost()}`);

coffee = new MilkDecorator(coffee);
console.log(`${coffee.getDescription()} ${coffee.getCost()}`);

coffee = new SugarDecorator(coffee);
console.log(`${coffee.getDescription()} ${coffee.getCost()}`);

coffee = new WhippedCreamDecorator(coffee);
console.log(`${coffee.getDescription()} ${coffee.getCost()}`);

console.log();

// Example 2: Text Formatting
console.log('--- Text Formatting Example ---');

let text: TextComponent = new SimpleText('Hello World');
console.log(`Original: ${text.getText()}`);

text = new BoldDecorator(text);
console.log(`Bold: ${text.getText()}`);

text = new ItalicDecorator(text);
console.log(`Bold + Italic: ${text.getText()}`);

text = new UnderlineDecorator(text);
console.log(`Bold + Italic + Underline: ${text.getText()}`);

console.log();

// Example 3: Data Stream Processing
console.log('--- Data Stream Processing Example ---');

let dataSource: DataSource = new FileDataSource();
console.log(`\nWriting plain data:`);
dataSource.writeData('Important Data');
console.log(`Reading: ${dataSource.readData()}`);

console.log(`\nWriting encrypted data:`);
dataSource = new EncryptionDecorator(dataSource);
dataSource.writeData('Secret Data');
console.log(`Reading: ${dataSource.readData()}`);

console.log(`\nWriting encrypted and compressed data:`);
dataSource = new CompressionDecorator(dataSource);
dataSource.writeData('Very Secret Data');
console.log(`Reading: ${dataSource.readData()}`);
}

// Run the demo
demoDecorator();
```

Facade

Description: Provides a **simplified, unified interface** to a complex subsystem or set of interfaces.

- **Why it is used:** To hide complexity from the client. The client interacts with a simple interface instead of dealing with multiple complex components.
- **Where it is commonly used:** API wrappers, library interfaces, startup/shutdown systems in complex applications.

Code Example (JAVASCRIPT)

```
/*
 * FACADE DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Facade pattern provides a simplified interface to a library, a
 * framework, or any other complex set of classes.
 * It hides the complexities of the larger system and provides a simpler
 * interface to the client.
 *
 * WHY DOES IT EXIST?
 * - When you need to provide a simple interface to a complex subsystem
 * - When you want to reduce dependencies between clients and the
 * implementation of an abstraction
 * - When you want to layer your subsystems (use facade to define entry
 * points)
 *
 * WHEN TO USE IT?
 * - When you have a complex system with many interacting classes
 * - When you want to provide a simple interface for clients to interact
 * with the system
 * - When you want to decouple the client from the subsystem implementation
 * - Common use cases: API libraries, database operations, complex UI
 * systems, payment processing
 *
 * PROS:
 * - Simplifies the interface to a complex subsystem
 * - Decouples clients from the subsystem (reduces coupling)
 * - Makes the system easier to use and understand
 * - Promotes loose coupling between subsystems and clients
 *
 * CONS:
 * - Can become a "god object" if not carefully designed
 * - May limit the flexibility of the subsystem
 * - Can hide useful functionality from advanced users
 * - Adds an extra layer of indirection
 *
 * REAL-WORLD ANALOGY:
 * Think of a car dashboard. The dashboard (facade) provides a simple
 * interface to the complex car systems (engine, transmission, brakes, etc.).
 * You don't need to know how the engine works to drive the car - you just use
```

the steering wheel, pedals, and dashboard controls. The dashboard hides the complexity while giving you access to the essential functions.

```
*/

// Subsystem Class 1: CPU
class CPU {
  freeze(): void {
    console.log('CPU: Freezing...');
  }

  jump(position: number): void {
    console.log(`CPU: Jumping to position ${position}`);
  }

  execute(): void {
    console.log('CPU: Executing instructions...');
  }
}

// Subsystem Class 2: Memory
class Memory {
  load(position: number, data: string): void {
    console.log(`Memory: Loading data '${data}' at position ${position}`);
  }
}

// Subsystem Class 3: Hard Drive
class HardDrive {
  read(lba: number, size: number): string {
    console.log(`HardDrive: Reading ${size} bytes from LBA ${lba}`);
    return 'Boot Data';
  }
}

// Facade - provides a simplified interface to the complex subsystem
class ComputerFacade {
  private cpu: CPU;
  private memory: Memory;
  private hardDrive: HardDrive;

  constructor() {
    this.cpu = new CPU();
    this.memory = new Memory();
    this.hardDrive = new HardDrive();
  }

  start(): void {
    console.log('=== Starting Computer ===');
    this.cpu.freeze();
    this.memory.load(0, this.hardDrive.read(0, 1024));
    this.cpu.jump(0);
    this.cpu.execute();
    console.log('=== Computer Started ===');
  }
}
```

```
shutdown(): void {
  console.log('=== Shutting Down Computer ===');
  console.log('CPU: Stopping...');
  console.log('Memory: Clearing...');
  console.log('HardDrive: Spinning down...');
  console.log('=== Computer Shutdown ===');
}

// Example 2: Home Theater System
// Shows how Facade can simplify controlling multiple devices

class Amplifier {
  on(): void { console.log('Amplifier: ON'); }
  off(): void { console.log('Amplifier: OFF'); }
  setVolume(level: number): void { console.log(`Amplifier: Volume set to ${level}`); }
}

class Tuner {
  on(): void { console.log('Tuner: ON'); }
  off(): void { console.log('Tuner: OFF'); }
  setFrequency(freq: number): void { console.log(`Tuner: Frequency set to ${freq} FM`); }
}

class DVDPlayer {
  on(): void { console.log('DVD Player: ON'); }
  off(): void { console.log('DVD Player: OFF'); }
  play(movie: string): void { console.log(`DVD Player: Playing '${movie}'`); }
}

class Projector {
  on(): void { console.log('Projector: ON'); }
  off(): void { console.log('Projector: OFF'); }
  setInput(input: string): void { console.log(`Projector: Input set to ${input}`); }
}

class TheaterLights {
  on(): void { console.log('Theater Lights: ON'); }
  off(): void { console.log('Theater Lights: OFF'); }
  dim(level: number): void { console.log(`Theater Lights: Dimmed to ${level}%`); }
}

class Screen {
  up(): void { console.log('Screen: UP'); }
  down(): void { console.log('Screen: DOWN'); }
}

class PopcornPopper {
```

```
on(): void { console.log('Popcorn Popper: ON'); }
off(): void { console.log('Popcorn Popper: OFF'); }
pop(): void { console.log('Popcorn Popper: Popping popcorn!'); }
}

// Facade for Home Theater
class HomeTheaterFacade {
  private amp: Amplifier;
  private tuner: Tuner;
  private dvd: DVDPlayer;
  private projector: Projector;
  private lights: TheaterLights;
  private screen: Screen;
  private popper: PopcornPopper;

  constructor() {
    this.amp = new Amplifier();
    this.tuner = new Tuner();
    this.dvd = new DVDPlayer();
    this.projector = new Projector();
    this.lights = new TheaterLights();
    this.screen = new Screen();
    this.popper = new PopcornPopper();
  }

  watchMovie(movie: string): void {
    console.log('\n=== Get ready to watch a movie ===');
    this.popper.on();
    this.popper.pop();
    this.lights.dim(10);
    this.screen.down();
    this.projector.on();
    this.projector.setInput('DVD');
    this.amp.on();
    this.amp.setVolume(5);
    this.dvd.on();
    this.dvd.play(movie);
    console.log('=== Movie is playing ===');
  }

  endMovie(): void {
    console.log('\n=== Shutting movie theater down ===');
    this.popper.off();
    this.lights.on();
    this.screen.up();
    this.projector.off();
    this.amp.off();
    this.dvd.off();
    console.log('=== Movie theater shut down ===');
  }

  listenToRadio(frequency: number): void {
    console.log('\n=== Tuning in to radio ===');
    this.amp.on();
  }
}
```

```
        this.amp.setVolume(5);
        this.tuner.on();
        this.tuner.setFrequency(frequency);
        console.log('=== Radio is playing ===');
    }

    turnOffRadio(): void {
        console.log('\n=== Shutting down radio ===');
        this.tuner.off();
        this.amp.off();
        console.log('=== Radio turned off ===');
    }
}

// Example 3: Order Processing System
// Shows how Facade can simplify complex business logic

class InventorySystem {
    checkInventory(item: string, quantity: number): boolean {
        console.log(`Inventory: Checking ${quantity} units of ${item}`);
        return true; // Simplified for demo
    }

    updateInventory(item: string, quantity: number): void {
        console.log(`Inventory: Updating ${quantity} units of ${item}`);
    }
}

class PaymentSystem {
    processPayment(amount: number): boolean {
        console.log(`Payment: Processing payment of $$${amount}`);
        return true; // Simplified for demo
    }

    refundPayment(amount: number): void {
        console.log(`Payment: Refunding $$${amount}`);
    }
}

class ShippingSystem {
    scheduleShipping(address: string): void {
        console.log(`Shipping: Scheduling delivery to ${address}`);
    }

    cancelShipping(): void {
        console.log('Shipping: Cancelling delivery');
    }
}

class NotificationSystem {
    sendConfirmation(email: string): void {
        console.log(`Notification: Sending confirmation to ${email}`);
    }
}
```

```
    sendCancellation(email: string): void {
        console.log(`Notification: Sending cancellation to ${email}`);
    }
}

// Facade for Order Processing
class OrderFacade {
    private inventory: InventorySystem;
    private payment: PaymentSystem;
    private shipping: ShippingSystem;
    private notification: NotificationSystem;

    constructor() {
        this.inventory = new InventorySystem();
        this.payment = new PaymentSystem();
        this.shipping = new ShippingSystem();
        this.notification = new NotificationSystem();
    }

    placeOrder(item: string, quantity: number, amount: number,
        address: string, email: string): boolean {
        console.log(`\n=== Processing Order ===`);

        if (!this.inventory.checkInventory(item, quantity)) {
            console.log('Order failed: Item not in stock');
            return false;
        }

        if (!this.payment.processPayment(amount)) {
            console.log('Order failed: Payment declined');
            return false;
        }

        this.inventory.updateInventory(item, -quantity);
        this.shipping.scheduleShipping(address);
        this.notification.sendConfirmation(email);

        console.log('=== Order Placed Successfully ===');
        return true;
    }

    cancelOrder(item: string, quantity: number, amount: number,
        email: string): void {
        console.log(`\n=== Cancelling Order ===`);
        this.inventory.updateInventory(item, quantity);
        this.payment.refundPayment(amount);
        this.shipping.cancelShipping();
        this.notification.sendCancellation(email);
        console.log('=== Order Cancelled ===');
    }
}

// Demo code
function demoFacade(): void {
```

```

console.log('=== Facade Pattern Demo ===\n');

// Example 1: Computer Boot
console.log('--- Computer Boot Example ---');

const computer = new ComputerFacade();
computer.start();
console.log();
computer.shutdown();

console.log();

// Example 2: Home Theater
console.log('--- Home Theater Example ---');

const homeTheater = new HomeTheaterFacade();
homeTheater.watchMovie('The Matrix');
homeTheater.endMovie();
homeTheater.listenToRadio(98.7);
homeTheater.turnOffRadio();

console.log();

// Example 3: Order Processing
console.log('--- Order Processing Example ---');

const orderSystem = new OrderFacade();
orderSystem.placeOrder('Laptop', 1, 999.99, '123 Main St',
'customer@email.com');
orderSystem.cancelOrder('Laptop', 1, 999.99, 'customer@email.com');
}

// Run the demo
demoFacade();

```

Flyweight

Description: Uses **sharing to efficiently support a large number of fine-grained objects**. It separates intrinsic (shared) state from extrinsic (unique) state.

- **Why it is used:** To reduce memory usage when a large number of similar objects need to be created.
- **Where it is commonly used:** Text editors (reusing character objects), game engines (reusing tile or bullet objects), rendering systems.

Code Example (JAVASCRIPT)

```

/*
 * FLYWEIGHT DESIGN PATTERN
 *
 * WHAT IS IT?

```

```

* The Flyweight pattern lets you fit more objects into the available RAM
by sharing common parts of state between multiple objects.
* It separates intrinsic state (shared) from extrinsic state (unique).
*
* WHY DOES IT EXIST?
* - When you need to reduce memory usage by sharing objects
* - When a large number of similar objects should be shared
* - When most of an object's state can be made extrinsic
*
* WHEN TO USE IT?
* - When your application uses a large number of objects
* - When storage costs are high because of the quantity of objects
* - When most of the object's state can be made extrinsic
* - When many groups of objects can be replaced by relatively few shared
objects
* - Common use cases: text editors (character formatting), game objects
(trees, soldiers), GUI elements (icons, fonts)
*
* PROS:
* - Reduces memory consumption by sharing objects
* - Improves performance when many similar objects are needed
* - Centralized state management
* - Can be combined with other patterns (Composite, Strategy)
*
* CONS:
* - May increase runtime complexity due to sharing
* - Requires careful identification of intrinsic vs extrinsic state
* - Can make code more complex
* - May sacrifice some encapsulation (extrinsic state passed to flyweight)
*
* REAL-WORLD ANALOGY:
* Think of a coffee shop. They don't make a new cup for every customer -
they reuse cups (intrinsic state: the cup itself). What changes is the
coffee inside (extrinsic state: the type of coffee). The cup is shared
(flyweight), while the coffee content varies per customer.
*/

// Flyweight Interface - defines the operations that can be performed on
flyweight objects
interface TreeType {
  draw(x: number, y: number): void;
  getName(): string;
  getColor(): string;
}

// Concrete Flyweight - implements the flyweight interface and stores
intrinsic state
class ConcreteTreeType implements TreeType {
  constructor(
    private name: string,
    private color: string,
    private texture: string
  ) {}

```

```
draw(x: number, y: number): void {
    console.log(`Drawing ${this.name} tree at (${x}, ${y}) with color
    ${this.color} and texture ${this.texture}`);
}

getName(): string {
    return this.name;
}

getColor(): string {
    return this.color;
}
}

// Flyweight Factory - creates and manages flyweight objects
class TreeFactory {
    private treeTypes: Map<string, TreeType> = new Map();

    // Helper function to create a unique key for the map
    private getKey(name: string, color: string, texture: string): string {
        return `${name}|${color}|${texture}`;
    }

    getTreeType(name: string, color: string, texture: string): TreeType {
        const key = this.getKey(name, color, texture);

        if (!this.treeTypes.has(key)) {
            console.log(`Creating new tree type: ${name}`);
            this.treeTypes.set(key, new ConcreteTreeType(name, color, texture));
        } else {
            console.log(`Reusing existing tree type: ${name}`);
        }

        return this.treeTypes.get(key)!;
    }

    getTotalTreeTypes(): number {
        return this.treeTypes.size;
    }
}

// Context - contains extrinsic state and uses flyweight objects
class Tree {
    constructor(
        private x: number,
        private y: number,
        private treeType: TreeType
    ) {}

    draw(): void {
        this.treeType.draw(this.x, this.y);
    }
}
```

```
// Example 2: Character Formatting in Text Editor
// Shows how Flyweight can be used for text rendering

interface CharacterFormat {
  apply(text: string): void;
  getFont(): string;
  getSize(): number;
  getColor(): string;
}

class ConcreteCharacterFormat implements CharacterFormat {
  constructor(
    private font: string,
    private size: number,
    private color: string,
    private bold: boolean,
    private italic: boolean
  ) {}

  apply(text: string): void {
    let output = `Rendering '${text}' with font: ${this.font}, size:
${this.size}, color: ${this.color}`;
    if (this.bold) output += ', bold';
    if (this.italic) output += ', italic';
    console.log(output);
  }

  getFont(): string {
    return this.font;
  }

  getSize(): number {
    return this.size;
  }

  getColor(): string {
    return this.color;
  }
}

class CharacterFormatFactory {
  private formats: Map<string, CharacterFormat> = new Map();

  private getKey(font: string, size: number, color: string, bold: boolean,
italic: boolean): string {
    return `${font}|${size}|${color}|${bold ? 'B' : ''}|${italic ? 'I' :
''}`;
  }

  getFormat(font: string, size: number, color: string, bold: boolean,
italic: boolean): CharacterFormat {
    const key = this.getKey(font, size, color, bold, italic);

    if (!this.formats.has(key)) {
```

```
        console.log(`Creating new format: ${font} ${size}pt`);
        this.formats.set(key, new ConcreteCharacterFormat(font, size, color,
bold, italic));
    } else {
        console.log(`Reusing existing format: ${font} ${size}pt`);
    }

    return this.formats.get(key)!;
}

getTotalFormats(): number {
    return this.formats.size;
}
}

class Character {
    constructor(
        private symbol: string,
        private format: CharacterFormat
    ) {}

    render(): void {
        this.format.apply(this.symbol);
    }
}

// Example 3: Game Units
// Shows how Flyweight can be used for game objects

interface SoldierType {
    render(x: number, y: number, weapon: string): void;
    getUniform(): string;
    getSpeed(): number;
}

class ConcreteSoldierType implements SoldierType {
    constructor(
        private uniform: string,
        private speed: number,
        private rank: string
    ) {}

    render(x: number, y: number, weapon: string): void {
        console.log(`Rendering ${this.rank} soldier at (${x}, ${y}) with
${this.uniform} uniform, speed ${this.speed}, holding ${weapon}`);
    }

    getUniform(): string {
        return this.uniform;
    }

    getSpeed(): number {
        return this.speed;
    }
}
```

```
}

class SoldierTypeFactory {
  private soldierTypes: Map<string, SoldierType> = new Map();

  private getKey(uniform: string, speed: number, rank: string): string {
    return `${uniform}|${speed}|${rank}`;
  }

  getSoldierType(uniform: string, speed: number, rank: string): SoldierType {
    const key = this.getKey(uniform, speed, rank);

    if (!this.soldierTypes.has(key)) {
      console.log(`Creating new soldier type: ${rank}`);
      this.soldierTypes.set(key, new ConcreteSoldierType(uniform, speed,
rank));
    } else {
      console.log(`Reusing existing soldier type: ${rank}`);
    }

    return this.soldierTypes.get(key)!;
  }

  getTotalSoldierTypes(): number {
    return this.soldierTypes.size;
  }
}

class Soldier {
  constructor(
    private x: number,
    private y: number,
    private weapon: string,
    private soldierType: SoldierType
  ) {}

  render(): void {
    this.soldierType.render(this.x, this.y, this.weapon);
  }

  move(newX: number, newY: number): void {
    this.x = newX;
    this.y = newY;
  }
}

// Demo code
function demoFlyweight(): void {
  console.log('=== Flyweight Pattern Demo ===\n');

  // Example 1: Forest Trees
  console.log('--- Forest Trees Example ---');
```

```
const treeFactory = new TreeFactory();

// Create trees with shared types
const forest: Tree[] = [];

forest.push(new Tree(10, 20, treeFactory.getTreeType('Oak', 'Green', 'Rough')));
forest.push(new Tree(30, 40, treeFactory.getTreeType('Oak', 'Green', 'Rough')));
forest.push(new Tree(50, 60, treeFactory.getTreeType('Oak', 'Green', 'Rough')));

forest.push(new Tree(15, 25, treeFactory.getTreeType('Pine', 'Dark Green', 'Smooth')));
forest.push(new Tree(35, 45, treeFactory.getTreeType('Pine', 'Dark Green', 'Smooth')));

forest.push(new Tree(20, 30, treeFactory.getTreeType('Oak', 'Green', 'Rough'))); // Reusing Oak

console.log('\nDrawing forest:');
for (const tree of forest) {
  tree.draw();
}

console.log(`\nTotal unique tree types: ${treeFactory.getTotalTreeTypes()}`);
console.log(`Total trees in forest: ${forest.length}`);

console.log();

// Example 2: Text Editor Characters
console.log('--- Text Editor Characters Example ---');

const formatFactory = new CharacterFormatFactory();

const document: Character[] = [];

document.push(new Character('H', formatFactory.getFormat('Arial', 12, 'Black', true, false)));
document.push(new Character('e', formatFactory.getFormat('Arial', 12, 'Black', false, false)));
document.push(new Character('l', formatFactory.getFormat('Arial', 12, 'Black', false, false)));
document.push(new Character('l', formatFactory.getFormat('Arial', 12, 'Black', false, false)));
document.push(new Character('o', formatFactory.getFormat('Arial', 12, 'Black', false, false)));

document.push(new Character(' ', formatFactory.getFormat('Arial', 12, 'Black', false, false)));

document.push(new Character('W', formatFactory.getFormat('Times New Roman', 14, 'Blue', true, true)));
```

```
document.push(new Character('o', formatFactory.getFormat('Times New Roman', 14, 'Blue', false, true)));
document.push(new Character('r', formatFactory.getFormat('Times New Roman', 14, 'Blue', false, true)));
document.push(new Character('l', formatFactory.getFormat('Times New Roman', 14, 'Blue', false, true)));
document.push(new Character('d', formatFactory.getFormat('Times New Roman', 14, 'Blue', false, true)));

console.log('\nRendering document:');
for (const character of document) {
  character.render();
}

console.log(`\nTotal unique formats:
${formatFactory.getTotalFormats()}`);
console.log(`Total characters: ${document.length}`);

console.log();

// Example 3: Game Soldiers
console.log('--- Game Soldiers Example ---');

const soldierFactory = new SoldierTypeFactory();

const army: Soldier[] = [];

army.push(new Soldier(10, 10, 'Rifle',
soldierFactory.getSoldierType('Camouflage', 5, 'Private')));
army.push(new Soldier(20, 20, 'Rifle',
soldierFactory.getSoldierType('Camouflage', 5, 'Private')));
army.push(new Soldier(30, 30, 'Rifle',
soldierFactory.getSoldierType('Camouflage', 5, 'Private')));

army.push(new Soldier(15, 15, 'Sniper',
soldierFactory.getSoldierType('Desert', 4, 'Sergeant')));
army.push(new Soldier(25, 25, 'Sniper',
soldierFactory.getSoldierType('Desert', 4, 'Sergeant')));

army.push(new Soldier(40, 40, 'Machine Gun',
soldierFactory.getSoldierType('Camouflage', 5, 'Private'))); // Reusing

console.log('\nRendering army:');
for (const soldier of army) {
  soldier.render();
}

console.log(`\nTotal unique soldier types:
${soldierFactory.getTotalSoldierTypes()}`);
console.log(`Total soldiers: ${army.length}`);
}
```

```
// Run the demo
demoFlyweight();
```

Proxy

Description: Provides a **substitute or placeholder** for another object to control access to it.

- **Why it is used:** To add a layer of control over the original object — for access control, lazy initialization, logging, or caching — without changing the original object.
- **Where it is commonly used:** Virtual proxies (lazy loading images), security proxies (access control), remote proxies (network calls).

Code Example (JAVASCRIPT)

```
/*
 * PROXY DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Proxy pattern provides a surrogate or placeholder object to control
access to another object.
 * It acts as an intermediary between the client and the real object.
 *
 * WHY DOES IT EXIST?
 * - When you need to control access to an object
 * - When you want to add additional functionality when accessing an object
 * - When you need to defer the creation or initialization of expensive
objects
 *
 * WHEN TO USE IT?
 * - When you want to add a layer of security to access an object
 * - When you need to cache results of expensive operations
 * - When you want to delay object creation until it's actually needed
(lazy initialization)
 * - When you need to log or monitor access to an object
 * - Common use cases: remote objects, virtual proxies, protection proxies,
smart references
 *
 * PROS:
 * - Can control access to the real object
 * - Can add additional functionality without changing the real object
 * - Can optimize performance (lazy loading, caching)
 * - Can provide security by controlling access
 *
 * CONS:
 * - Adds an extra layer of indirection
 * - Can increase response time due to proxy overhead
 * - Can make the code more complex
 * - May introduce bugs if the proxy doesn't perfectly mimic the real
object
 */
```

```
* REAL-WORLD ANALOGY:
* Think of a credit card. The credit card (proxy) represents your bank
account (real object). You don't carry your actual bank account around -
you use the card as a proxy. The card controls access to your money, adds
security (PIN, chip), and logs transactions, all while giving you access to
your funds.
*/

// Subject Interface - defines the common interface for RealSubject and
Proxy
interface Image {
    display(): void;
}

// RealSubject - the actual object that the proxy represents
class RealImage implements Image {
    private filename: string;

    constructor(filename: string) {
        this.filename = filename;
        this.loadFromDisk();
    }

    // Helper method to load the image from disk
    private loadFromDisk(): void {
        console.log(`Loading ${this.filename} from disk...`);
    }

    display(): void {
        console.log(`Displaying ${this.filename}`);
    }
}

// Proxy - controls access to the RealSubject
class ProxyImage implements Image {
    private filename: string;
    private realImage?: RealImage;

    constructor(filename: string) {
        this.filename = filename;
    }

    display(): void {
        // Lazy initialization: only create the real image when needed
        if (!this.realImage) {
            console.log('Creating real image on first access...');
            this.realImage = new RealImage(this.filename);
        }
        this.realImage.display();
    }
}

// Example 2: Protection Proxy
// Shows how Proxy can control access based on permissions
```

```
interface Database {
  query(sql: string): void;
}

class RealDatabase implements Database {
  query(sql: string): void {
    console.log(`Executing query: ${sql}`);
  }
}

class DatabaseProxy implements Database {
  private realDatabase?: RealDatabase;
  private currentUser: string;
  private permissions: Map<string, boolean> = new Map();

  constructor(user: string) {
    this.currentUser = user;
    // Set up permissions (in real app, this would come from auth system)
    this.permissions.set('admin', true);
    this.permissions.set('user', false);
    this.permissions.set('guest', false);
  }

  private hasPermission(): boolean {
    return this.permissions.get(this.currentUser) ?? false;
  }

  query(sql: string): void {
    if (this.hasPermission()) {
      if (!this.realDatabase) {
        this.realDatabase = new RealDatabase();
      }
      this.realDatabase.query(sql);
    } else {
      console.log(`Access denied: User '${this.currentUser}' does not have
permission to execute queries`);
    }
  }

  setUser(user: string): void {
    this.currentUser = user;
  }
}

// Example 3: Caching Proxy
// Shows how Proxy can cache expensive operations

interface Video {
  play(): void;
}

class RealVideo implements Video {
  private filename: string;
```

```
    constructor(filename: string) {
        this.filename = filename;
        this.loadVideo();
    }

    private loadVideo(): void {
        console.log(`Loading video ${this.filename} from server...`);
    }

    play(): void {
        console.log(`Playing video: ${this.filename}`);
    }
}

class VideoProxy implements Video {
    private filename: string;
    private realVideo?: RealVideo;
    private isLoading: boolean = false;

    constructor(filename: string) {
        this.filename = filename;
    }

    play(): void {
        if (!this.isLoading) {
            console.log('Loading video on first play...');
            this.realVideo = new RealVideo(this.filename);
            this.isLoading = true;
        } else {
            console.log('Using cached video...');
        }
        this.realVideo!.play();
    }
}

// Example 4: Logging Proxy
// Shows how Proxy can add logging functionality

interface Service {
    doWork(): void;
}

class RealService implements Service {
    doWork(): void {
        console.log('Service: Performing work...');
    }
}

class LoggingProxy implements Service {
    private realService: RealService;

    constructor() {
        this.realService = new RealService();
    }
}
```

```
}

private log(message: string): void {
    console.log(`[LOG] ${message}`);
}

doWork(): void {
    this.log('Before doWork');
    this.realService.doWork();
    this.log('After doWork');
}
}

// Example 5: Remote Proxy (simulated)
// Shows how Proxy can represent remote objects

interface RemoteServer {
    request(data: string): void;
}

class RealRemoteServer implements RemoteServer {
    request(data: string): void {
        console.log(`Remote server processing request: ${data}`);
    }
}

class RemoteServerProxy implements RemoteServer {
    private remoteServer?: RealRemoteServer;
    private serverAddress: string;

    constructor(address: string) {
        this.serverAddress = address;
    }

    private connectToServer(): void {
        console.log(`Connecting to remote server at ${this.serverAddress}...`);
    }

    request(data: string): void {
        if (!this.remoteServer) {
            this.connectToServer();
            this.remoteServer = new RealRemoteServer();
        }
        console.log('Proxy forwarding request to remote server...');
        this.remoteServer.request(data);
    }
}

// Demo code
function demoProxy(): void {
    console.log('=== Proxy Pattern Demo ===\n');

    // Example 1: Virtual Proxy (Lazy Loading)
    console.log('--- Virtual Proxy (Lazy Loading) Example ---');
```

```
const image1: Image = new ProxyImage('photo1.jpg');
const image2: Image = new ProxyImage('photo2.jpg');
const image3: Image = new ProxyImage('photo1.jpg'); // Same as image1

console.log('\nFirst display of image1:');
image1.display();

console.log('\nSecond display of image1 (should use cached):');
image1.display();

console.log('\nDisplay of image2:');
image2.display();

console.log('\nDisplay of image3 (same as image1, should use cached):');
image3.display();

console.log();

// Example 2: Protection Proxy
console.log('--- Protection Proxy Example ---');

const db = new DatabaseProxy('guest');
console.log('\nGuest trying to execute query:');
db.query('SELECT * FROM users');

console.log('\nSwitching to admin user:');
db.setUser('admin');
db.query('SELECT * FROM users');

console.log('\nSwitching back to regular user:');
db.setUser('user');
db.query('SELECT * FROM users');

console.log();

// Example 3: Caching Proxy
console.log('--- Caching Proxy Example ---');

const video = new VideoProxy('movie.mp4');

console.log('\nFirst play (will load):');
video.play();

console.log('\nSecond play (will use cache):');
video.play();

console.log('\nThird play (will use cache):');
video.play();

console.log();

// Example 4: Logging Proxy
console.log('--- Logging Proxy Example ---');
```

```
const loggingService = new LoggingProxy();
loggingService.doWork();

console.log();

// Example 5: Remote Proxy
console.log('--- Remote Proxy Example ---');

const remoteProxy = new RemoteServerProxy('192.168.1.100');

console.log('\nFirst request (will connect):');
remoteProxy.request('GET /api/data');

console.log('\nSecond request (already connected):');
remoteProxy.request('POST /api/data');
}

// Run the demo
demoProxy();
```

3. Behavioral Design Patterns

Overview

Behavioral patterns deal with **how objects interact and communicate** with each other. They focus on the assignment of responsibilities between objects and define clear communication patterns to make the system more flexible in carrying out behavior.

Chain of Responsibility

Description: Passes a request along a **chain of handlers**, where each handler decides to process the request or pass it to the next handler in the chain.

- **Why it is used:** To decouple the sender of a request from its receivers and give multiple objects the chance to handle the request.
- **Where it is commonly used:** Middleware pipelines (HTTP request handling), event handling systems, logging with multiple levels.

Code Example (JAVASCRIPT)

```
/*
 * CHAIN OF RESPONSIBILITY DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Chain of Responsibility pattern lets you pass requests along a chain
of handlers.
 * Each handler decides either to process the request or to pass it to the
```

```

next handler in the chain.
*
* WHY DOES IT EXIST?
* - When you want to decouple senders of a request from its receivers
* - When multiple objects can handle a request without the sender knowing
which one will
* - When you want to dynamically add or remove handlers at runtime
*
* WHEN TO USE IT?
* - When more than one object can handle a request, and the handler isn't
known a priori
* - When you want to issue a request to one of several objects without
specifying the receiver explicitly
* - When the set of objects that can handle a request should be specified
dynamically
* - Common use cases: event handling systems, logging frameworks, approval
workflows, help systems
*
* PROS:
* - Decouples sender from receiver (sender doesn't know which handler will
process)
* - Open/Closed Principle (can add new handlers without changing existing
code)
* - Can control the order of handling
* - Simplifies object interactions
*
* CONS:
* - No guarantee that a request will be handled
* - Can lead to deep chains that are hard to debug
* - May affect performance if the chain is long
* - Can be difficult to observe runtime characteristics
*
* REAL-WORLD ANALOGY:
* Think of a customer service call center. When you call, your request
goes through a chain of representatives. If the first representative can't
help, they transfer you to a specialist. If that specialist can't help,
they transfer you to a manager. The call moves up the chain until someone
can help or you reach the end.
*/

// Handler interface - declares the handling method and sets a reference to
the next handler
abstract class Handler {
  protected nextHandler: Handler | null = null;

  // Set the next handler in the chain
  setNext(handler: Handler): Handler {
    this.nextHandler = handler;
    return handler;
  }

  // Handle the request - to be implemented by concrete handlers
  abstract handle(request: string): void;

```

```

// Pass the request to the next handler if it exists
protected passRequest(request: string): void {
  if (this.nextHandler) {
    this.nextHandler.handle(request);
  } else {
    console.log('Request could not be handled');
  }
}
}

// Concrete Handler 1: Support Agent
class SupportAgent extends Handler {
  handle(request: string): void {
    if (request === 'technical_issue' || request === 'basic_question') {
      console.log(`SupportAgent: Handling request - ${request}`);
    } else {
      console.log('SupportAgent: Passing to next handler...');
      this.passRequest(request);
    }
  }
}

// Concrete Handler 2: Technical Specialist
class TechnicalSpecialist extends Handler {
  handle(request: string): void {
    if (request === 'complex_technical' || request === 'bug_report') {
      console.log(`TechnicalSpecialist: Handling request - ${request}`);
    } else {
      console.log('TechnicalSpecialist: Passing to next handler...');
      this.passRequest(request);
    }
  }
}

// Concrete Handler 3: Manager
class Manager extends Handler {
  handle(request: string): void {
    if (request === 'refund' || request === 'complaint' || request ===
'escalation') {
      console.log(`Manager: Handling request - ${request}`);
    } else {
      console.log('Manager: Passing to next handler...');
      this.passRequest(request);
    }
  }
}

// Example 2: Document Approval Chain
// Shows how Chain of Responsibility can be used for approval workflows

abstract class Approver {
  protected nextApprover: Approver | null = null;

  setNext(approver: Approver): Approver {

```

```
    this.nextApprover = approver;
    return approver;
}

abstract approveRequest(amount: number): void;

protected passRequest(amount: number): void {
    if (this.nextApprover) {
        this.nextApprover.approveRequest(amount);
    } else {
        console.log('Request cannot be approved');
    }
}
}

class TeamLead extends Approver {
    approveRequest(amount: number): void {
        if (amount <= 1000) {
            console.log(`TeamLead: Approved request of ${amount}`);
        } else {
            console.log('TeamLead: Amount exceeds limit, passing to manager...');
            this.passRequest(amount);
        }
    }
}

class ManagerApprover extends Approver {
    approveRequest(amount: number): void {
        if (amount <= 5000) {
            console.log(`Manager: Approved request of ${amount}`);
        } else {
            console.log('Manager: Amount exceeds limit, passing to director...');
            this.passRequest(amount);
        }
    }
}

class Director extends Approver {
    approveRequest(amount: number): void {
        if (amount <= 20000) {
            console.log(`Director: Approved request of ${amount}`);
        } else {
            console.log('Director: Amount exceeds limit, passing to CEO...');
            this.passRequest(amount);
        }
    }
}

class CEO extends Approver {
    approveRequest(amount: number): void {
        console.log(`CEO: Approved request of ${amount} (final approver)`);
    }
}
```

```
// Example 3: Logging Chain
// Shows how Chain of Responsibility can be used for logging

abstract class Logger {
  protected nextLogger: Logger | null = null;

  setNext(logger: Logger): Logger {
    this.nextLogger = logger;
    return logger;
  }

  abstract log(message: string, severity: number): void;

  protected passLog(message: string, severity: number): void {
    if (this.nextLogger) {
      this.nextLogger.log(message, severity);
    }
  }
}

class ConsoleLogger extends Logger {
  log(message: string, severity: number): void {
    if (severity >= 1) {
      console.log(`[CONSOLE] ${message}`);
    }
    this.passLog(message, severity);
  }
}

class FileLogger extends Logger {
  log(message: string, severity: number): void {
    if (severity >= 2) {
      console.log(`[FILE] ${message}`);
    }
    this.passLog(message, severity);
  }
}

class ErrorLogger extends Logger {
  log(message: string, severity: number): void {
    if (severity >= 3) {
      console.log(`[ERROR] ${message}`);
    }
    this.passLog(message, severity);
  }
}

// Demo code
console.log('=== Chain of Responsibility Pattern Demo ===\n');

// Example 1: Customer Service Chain
console.log('--- Customer Service Chain ---');

const agent = new SupportAgent();
```

```
const specialist = new TechnicalSpecialist();
const manager = new Manager();

agent.setNext(specialist).setNext(manager);

console.log('\nRequest: basic_question');
agent.handle('basic_question');

console.log('\nRequest: complex_technical');
agent.handle('complex_technical');

console.log('\nRequest: refund');
agent.handle('refund');

console.log('\nRequest: unknown');
agent.handle('unknown');

console.log('');

// Example 2: Document Approval Chain
console.log('--- Document Approval Chain ---');

const teamLead = new TeamLead();
const manager2 = new ManagerApprover();
const director = new Director();
const ceo = new CEO();

teamLead.setNext(manager2).setNext(director).setNext(ceo);

console.log('\nRequest: $500');
teamLead.approveRequest(500);

console.log('\nRequest: $3000');
teamLead.approveRequest(3000);

console.log('\nRequest: $15000');
teamLead.approveRequest(15000);

console.log('\nRequest: $50000');
teamLead.approveRequest(50000);

console.log('');

// Example 3: Logging Chain
console.log('--- Logging Chain ---');

const consoleLogger = new ConsoleLogger();
const fileLogger = new FileLogger();
const errorLogger = new ErrorLogger();

consoleLogger.setNext(fileLogger).setNext(errorLogger);

console.log('\nLog: Info message (severity 1)');
consoleLogger.log('Info message', 1);
```

```
console.log('\nLog: Warning message (severity 2)');
consoleLogger.log('Warning message', 2);

console.log('\nLog: Error message (severity 3)');
consoleLogger.log('Error message', 3);
```

Command

Description: Encapsulates a request as an object, allowing you to parameterize clients with different requests, queue operations, and support undoable actions.

- **Why it is used:** To decouple the object that sends a request from the object that executes it, enabling flexible command management.
- **Where it is commonly used:** Undo/redo systems, task schedulers, GUI buttons and menu actions, transactional operations.

Code Example (JAVASCRIPT)

```
/*
 * COMMAND DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Command pattern turns a request into a stand-alone object that
contains all information about the request.
 * It lets you parameterize clients with different requests, queue or log
requests, and support undoable operations.
 *
 * WHY DOES IT EXIST?
 * - When you want to parameterize objects with operations
 * - When you want to queue operations, schedule their execution, or
execute them remotely
 * - When you need to support undo operations
 * - When you want to decouple the object that invokes an operation from
the one that knows how to perform it
 *
 * WHEN TO USE IT?
 * - When you want to parameterize objects with an operation to execute
 * - When you want to queue operations, schedule their execution, or
execute them remotely
 * - When you need to support undo/redo functionality
 * - When you want to decouple the invoker from the receiver
 * - Common use cases: GUI buttons and menus, multi-level undo, macro
recording, transactional systems
 *
 * PROS:
 * - Decouples the object that invokes the operation from the one that
knows how to perform it
 * - Can add new commands without changing existing code (Open/Closed
Principle)
```

```
* - Allows you to assemble commands into more complex commands (composite
pattern)
* - Supports undo/redo operations
*
* CONS:
* - Can lead to many small command classes (code complexity)
* - Can complicate the code if the receiver and command are tightly
coupled
* - May introduce additional layers of indirection
* - Can make debugging more difficult
*
* REAL-WORLD ANALOGY:
* Think of a restaurant order. The customer (client) places an order
(command) with the waiter (invoker). The waiter writes down the order and
gives it to the kitchen (receiver). The kitchen executes the command
(prepare the meal). The order can be cancelled (undo) before it's
prepared, and multiple orders can be queued.
*/

// Receiver - knows how to perform the operations
class Light {
  on(): void {
    console.log('Light is ON');
  }

  off(): void {
    console.log('Light is OFF');
  }
}

class Stereo {
  on(): void {
    console.log('Stereo is ON');
  }

  off(): void {
    console.log('Stereo is OFF');
  }

  setVolume(level: number): void {
    console.log(`Stereo volume set to ${level}`);
  }
}

// Command interface - declares the execute method
interface Command {
  execute(): void;
  undo(): void;
}

// Concrete Command 1: Light On
class LightOnCommand implements Command {
  private light: Light;
```

```
    constructor(light: Light) {
        this.light = light;
    }

    execute(): void {
        this.light.on();
    }

    undo(): void {
        this.light.off();
    }
}

// Concrete Command 2: Light Off
class LightOffCommand implements Command {
    private light: Light;

    constructor(light: Light) {
        this.light = light;
    }

    execute(): void {
        this.light.off();
    }

    undo(): void {
        this.light.on();
    }
}

// Concrete Command 3: Stereo On with Volume
class StereoOnCommand implements Command {
    private stereo: Stereo;
    private previousVolume: number = 0;

    constructor(stereo: Stereo) {
        this.stereo = stereo;
    }

    execute(): void {
        this.stereo.on();
        this.stereo.setVolume(10);
        this.previousVolume = 10;
    }

    undo(): void {
        this.stereo.setVolume(this.previousVolume);
        this.stereo.off();
    }
}

// Concrete Command 4: Stereo Off
class StereoOffCommand implements Command {
    private stereo: Stereo;
```

```
    constructor(stereo: Stereo) {
        this.stereo = stereo;
    }

    execute(): void {
        this.stereo.off();
    }

    undo(): void {
        this.stereo.on();
    }
}

// Invoker - asks the command to carry out the request
class RemoteControl {
    private commandHistory: Command[] = [];
    private undoStack: Command[] = [];

    setCommand(command: Command): void {
        this.commandHistory.push(command);
    }

    buttonPressed(slot: number): void {
        if (slot < this.commandHistory.length) {
            this.commandHistory[slot].execute();
            this.undoStack.push(this.commandHistory[slot]);
        }
    }

    undoButtonPressed(): void {
        if (this.undoStack.length > 0) {
            const command = this.undoStack.pop()!;
            command.undo();
        } else {
            console.log('Nothing to undo');
        }
    }
}

// Example 2: Macro Command
// Shows how multiple commands can be combined into one

class MacroCommand implements Command {
    private commands: Command[] = [];

    addCommand(command: Command): void {
        this.commands.push(command);
    }

    execute(): void {
        for (const command of this.commands) {
            command.execute();
        }
    }
}
```

```
}

undo(): void {
  // Undo in reverse order
  for (let i = this.commands.length - 1; i >= 0; i--) {
    this.commands[i].undo();
  }
}

// Example 3: Transaction Command
// Shows how Command can be used for transactional operations

class BankAccount {
  private balance: number;

  constructor(initialBalance: number = 0) {
    this.balance = initialBalance;
  }

  deposit(amount: number): void {
    this.balance += amount;
    console.log(`Deposited ${amount}. New balance: ${this.balance}`);
  }

  withdraw(amount: number): void {
    if (this.balance >= amount) {
      this.balance -= amount;
      console.log(`Withdrew ${amount}. New balance: ${this.balance}`);
    } else {
      console.log('Insufficient funds');
    }
  }

  getBalance(): number {
    return this.balance;
  }
}

class DepositCommand implements Command {
  private account: BankAccount;
  private amount: number;

  constructor(account: BankAccount, amount: number) {
    this.account = account;
    this.amount = amount;
  }

  execute(): void {
    this.account.deposit(this.amount);
  }

  undo(): void {
    this.account.withdraw(this.amount);
  }
}
```

```
}  
}  
  
class WithdrawCommand implements Command {  
  private account: BankAccount;  
  private amount: number;  
  private success: boolean = false;  
  
  constructor(account: BankAccount, amount: number) {  
    this.account = account;  
    this.amount = amount;  
  }  
  
  execute(): void {  
    const oldBalance = this.account.getBalance();  
    this.account.withdraw(this.amount);  
    this.success = this.account.getBalance() < oldBalance;  
  }  
  
  undo(): void {  
    if (this.success) {  
      this.account.deposit(this.amount);  
    }  
  }  
}  
  
// Demo code  
console.log('=== Command Pattern Demo ===\n');  
  
// Example 1: Remote Control  
console.log('--- Remote Control Example ---');  
  
const livingRoomLight = new Light();  
const livingRoomStereo = new Stereo();  
  
const lightOn = new LightOnCommand(livingRoomLight);  
const lightOff = new LightOffCommand(livingRoomLight);  
const stereoOn = new StereoOnCommand(livingRoomStereo);  
const stereoOff = new StereoOffCommand(livingRoomStereo);  
  
const remote = new RemoteControl();  
remote.setCommand(lightOn);  
remote.setCommand(lightOff);  
remote.setCommand(stereoOn);  
remote.setCommand(stereoOff);  
  
console.log('\nTurning on light (slot 0):');  
remote.buttonPressed(0);  
  
console.log('\nTurning on stereo (slot 2):');  
remote.buttonPressed(2);  
  
console.log('\nUndo last action:');  
remote.undoButtonPressed();
```

```
console.log('\nTurning off light (slot 1):');
remote.buttonPressed(1);

console.log('\nUndo last action:');
remote.undoButtonPressed();

console.log('');

// Example 2: Macro Command
console.log('--- Macro Command Example ---');

const partyMode = new MacroCommand();
partyMode.addCommand(lightOn);
partyMode.addCommand(stereoOn);

const shutdownMode = new MacroCommand();
shutdownMode.addCommand(stereoOff);
shutdownMode.addCommand(lightOff);

console.log('\nActivating party mode:');
partyMode.execute();

console.log('\nActivating shutdown mode:');
shutdownMode.execute();

console.log('\nUndo shutdown:');
shutdownMode.undo();

console.log('');

// Example 3: Transaction Command
console.log('--- Transaction Command Example ---');

const myAccount = new BankAccount(1000);

console.log('\nInitial balance: $' + myAccount.getBalance());

const deposit = new DepositCommand(myAccount, 500);
const withdraw = new WithdrawCommand(myAccount, 200);

console.log('\nExecuting deposit:');
deposit.execute();

console.log('\nExecuting withdrawal:');
withdraw.execute();

console.log('\nUndo withdrawal:');
withdraw.undo();

console.log('\nFinal balance: $' + myAccount.getBalance());
```

Iterator

Description: Provides a way to **sequentially access elements** of a collection without exposing its underlying representation.

- **Why it is used:** To traverse different types of collections in a uniform way without depending on their internal structure.
 - **Where it is commonly used:** Iterating over lists, trees, graphs, and custom data structures in any programming language.
-

Code Example (JAVASCRIPT)

```
/*
 * ITERATOR DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Iterator pattern provides a way to access the elements of an
 * aggregate object sequentially without exposing its underlying
 * representation.
 * It traverses the elements of a collection without knowing its internal
 * structure.
 *
 * WHY DOES IT EXIST?
 * - When you want to access a collection's contents without exposing its
 * internal representation
 * - When you want to provide multiple ways to traverse a collection
 * - When you want to provide a uniform interface for traversing different
 * data structures
 *
 * WHEN TO USE IT?
 * - When you have a collection of objects and want to traverse it without
 * exposing its internal structure
 * - When you want to provide multiple traversal algorithms for the same
 * collection
 * - When you want to simplify the interface for accessing a complex data
 * structure
 * - Common use cases: collections, tree structures, database result sets,
 * file systems
 *
 * PROS:
 * - Single Responsibility Principle (separates traversal logic from the
 * collection)
 * - Open/Closed Principle (can add new iterators without changing the
 * collection)
 * - Provides a uniform interface for traversing different collections
 * - Can support multiple traversals simultaneously
 *
 * CONS:
 * - Can be overkill for simple collections
 * - May violate encapsulation if the iterator exposes too much about the
 * collection
 */
```

```
* - Can be less efficient than direct access
* - May complicate the code for simple use cases
*
* REAL-WORLD ANALOGY:
* Think of a TV remote control with channel up/down buttons. You don't
need to know how the TV stores channels - you just use the next/previous
buttons to navigate. The remote (iterator) provides a simple interface to
traverse the channels (collection) without knowing the internal channel
storage mechanism.
*/

// Iterator interface - declares the operations for traversing
interface Iterator<T> {
    hasNext(): boolean;
    next(): T;
}

// Aggregate interface - declares the method for creating an iterator
interface Aggregate<T> {
    createIterator(): Iterator<T>;
}

// Concrete Aggregate - implements the aggregate interface
class BookCollection implements Aggregate<string> {
    private items: string[] = [];

    add(item: string): void {
        this.items.push(item);
    }

    createIterator(): Iterator<string> {
        return new BookIterator(this);
    }

    size(): number {
        return this.items.length;
    }

    get(index: number): string {
        return this.items[index];
    }
}

// Concrete Iterator
class BookIterator implements Iterator<string> {
    private collection: BookCollection;
    private position: number = 0;

    constructor(collection: BookCollection) {
        this.collection = collection;
    }

    hasNext(): boolean {
        return this.position < this.collection.size();
    }
}
```

```
}

next(): string {
  if (this.hasNext()) {
    return this.collection.get(this.position++);
  }
  return '';
}
}

// Example 2: Tree Traversal Iterator
// Shows how Iterator can be used for tree structures

class TreeNode<T> {
  data: T;
  left: TreeNode<T> | null = null;
  right: TreeNode<T> | null = null;

  constructor(data: T) {
    this.data = data;
  }
}

class BinaryTreeIterator<T> implements Iterator<T> {
  private stack: TreeNode<T>[] = [];

  constructor(root: TreeNode<T> | null) {
    this.pushLeft(root);
  }

  private pushLeft(node: TreeNode<T> | null): void {
    while (node) {
      this.stack.push(node);
      node = node.left;
    }
  }

  hasNext(): boolean {
    return this.stack.length > 0;
  }

  next(): T {
    if (!this.hasNext()) return null as T;

    const node = this.stack.pop()!;

    if (node.right) {
      this.pushLeft(node.right);
    }

    return node.data;
  }
}
```

```
// Example 3: Custom Container with Iterator
// Shows how to implement iterator for a custom data structure

class CustomList<T> implements Aggregate<T> {
  private head: Node<T> | null = null;
  private count: number = 0;

  add(item: T): void {
    const newNode = new Node(item);
    if (!this.head) {
      this.head = newNode;
    } else {
      let current = this.head;
      while (current.next) {
        current = current.next;
      }
      current.next = newNode;
    }
    this.count++;
  }

  createIterator(): Iterator<T> {
    return new ListIterator(this.head);
  }
}

class Node<T> {
  data: T;
  next: Node<T> | null = null;

  constructor(data: T) {
    this.data = data;
  }
}

class ListIterator<T> implements Iterator<T> {
  private current: Node<T> | null;

  constructor(head: Node<T> | null) {
    this.current = head;
  }

  hasNext(): boolean {
    return this.current !== null;
  }

  next(): T {
    if (!this.hasNext()) return null as T;
    const data = this.current!.data;
    this.current = this.current!.next;
    return data;
  }
}
```

```
// Demo code
console.log('=== Iterator Pattern Demo ===\n');

// Example 1: Book Collection
console.log('--- Book Collection Example ---');

const books = new BookCollection();
books.add('Design Patterns');
books.add('Clean Code');
books.add('Refactoring');
books.add('The Pragmatic Programmer');

const bookIterator = books.createIterator();

console.log('\nIterating through books:');
while (bookIterator.hasNext()) {
    console.log(' - ' + bookIterator.next());
}

console.log('');

// Example 2: Binary Tree Traversal
console.log('--- Binary Tree Traversal Example ---');

// Build a simple binary tree
const root = new TreeNode(5);
root.left = new TreeNode(3);
root.right = new TreeNode(7);
root.left.left = new TreeNode(1);
root.left.right = new TreeNode(4);
root.right.left = new TreeNode(6);
root.right.right = new TreeNode(9);

const treeIterator = new BinaryTreeIterator(root);

console.log('\nIn-order traversal:');
while (treeIterator.hasNext()) {
    console.log(' ' + treeIterator.next());
}

console.log('');

// Example 3: Custom List Iterator
console.log('--- Custom List Iterator Example ---');

const numbers = new CustomList<number>();
numbers.add(10);
numbers.add(20);
numbers.add(30);
numbers.add(40);
numbers.add(50);

const listIterator = numbers.createIterator();
```

```
console.log('\nIterating through custom list:');
while (listIterator.hasNext()) {
    console.log(' ' + listIterator.next());
}
```

Mediator

Description: Defines an object that **encapsulates how a set of objects interact**. It promotes loose coupling by preventing objects from referring to each other directly.

- **Why it is used:** To reduce the chaotic dependencies between many objects communicating directly. All communication goes through the mediator.
- **Where it is commonly used:** Chat room systems, air traffic control systems, UI dialog form coordination.

Code Example (JAVASCRIPT)

```
/*
 * MEDIATOR DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Mediator pattern defines an object that encapsulates how a set of
objects interact.
 * It promotes loose coupling by preventing objects from referring to each
other explicitly.
 *
 * WHY DOES IT EXIST?
 * - When you want to reduce the chaotic dependencies between objects
 * - When you want to centralize complex communications and control logic
between related objects
 * - When you want to reuse the interaction logic without duplicating it
across multiple objects
 *
 * WHEN TO USE IT?
 * - When a set of objects communicate in well-defined but complex ways
 * - When you want to customize a behavior that's distributed between
several objects without subclassing
 * - When you have many objects that need to interact with each other
 * - Common use cases: chat rooms, air traffic control, GUI frameworks,
auction systems
 *
 * PROS:
 * - Single Responsibility Principle (extracts communication logic to a
separate class)
 * - Open/Closed Principle (can add new colleagues without changing the
mediator)
 * - Reduces coupling between colleagues
 * - Simplifies object interactions
 *
 * CONS:
```

```
* - The mediator can become a monolith (god object) if not carefully
designed
* - Can make the system harder to understand due to indirect communication
* - May reduce performance due to indirection
* - Can be overkill for simple interactions
*
* REAL-WORLD ANALOGY:
* Think of an air traffic control tower. Planes (colleagues) don't
communicate directly with each other - they all communicate through the
tower (mediator). The tower coordinates takeoffs, landings, and routes,
ensuring planes don't collide. This centralizes the coordination logic and
prevents planes from needing to know about each other.
*/

// Mediator interface - declares the communication interface
interface Mediator {
  sendMessage(message: string, sender: string): void;
}

// Colleague interface - declares the interface for communicating with the
mediator
abstract class Colleague {
  protected mediator: Mediator;
  protected name: string;

  constructor(mediator: Mediator, name: string) {
    this.mediator = mediator;
    this.name = name;
  }

  send(message: string): void {
    this.mediator.sendMessage(message, this.name);
  }

  abstract receive(message: string): void;

  getName(): string {
    return this.name;
  }
}

// Concrete Colleague 1
class User extends Colleague {
  constructor(mediator: Mediator, name: string) {
    super(mediator, name);
  }

  receive(message: string): void {
    console.log(`${this.name} received: ${message}`);
  }
}

// Concrete Colleague 2
class Bot extends Colleague {
```

```
constructor(mediator: Mediator, name: string) {
  super(mediator, name);
}

receive(message: string): void {
  console.log(`${this.name} received: ${message}`);
  // Bot can auto-reply
  this.send('Auto-reply: Thanks for your message!');
}
}

// Concrete Mediator
class ChatRoom implements Mediator {
  private colleagues: Map<string, Colleague> = new Map();

  addUser(colleague: Colleague): void {
    this.colleagues.set(colleague.getName(), colleague);
  }

  sendMessage(message: string, sender: string): void {
    console.log(`${sender} sends: ${message}`);
    for (const [name, colleague] of this.colleagues) {
      if (name !== sender) {
        colleague.receive(message);
      }
    }
  }
}

// Example 2: Air Traffic Control
// Shows how Mediator can coordinate complex interactions

class Aircraft {
  protected controlTower: Mediator;
  protected flightNumber: string;
  protected altitude: number = 0;
  protected speed: number = 0;

  constructor(tower: Mediator, flightNumber: string) {
    this.controlTower = tower;
    this.flightNumber = flightNumber;
  }

  requestTakeoff(): void {
    console.log(`${this.flightNumber} requesting takeoff clearance...`);
    // In real implementation, would send to mediator
  }

  requestLanding(): void {
    console.log(`${this.flightNumber} requesting landing clearance...`);
    // In real implementation, would send to mediator
  }

  setAltitude(alt: number): void {
```

```
        this.altitude = alt;
        console.log(`${this.flightNumber} at altitude ${this.altitude} feet`);
    }

    getFlightNumber(): string {
        return this.flightNumber;
    }
}

class ControlTower implements Mediator {
    private aircrafts: Map<string, Aircraft> = new Map();

    registerAircraft(aircraft: Aircraft): void {
        this.aircrafts.set(aircraft.getFlightNumber(), aircraft);
    }

    sendMessage(message: string, sender: string): void {
        // Simplified for demo
        console.log(`Control Tower: ${message} from ${sender}`);
    }

    grantTakeoff(flightNumber: string): void {
        console.log(`Control Tower: Takeoff granted to ${flightNumber}`);
        const aircraft = this.aircrafts.get(flightNumber);
        if (aircraft) {
            aircraft.setAltitude(10000);
        }
    }

    grantLanding(flightNumber: string): void {
        console.log(`Control Tower: Landing granted to ${flightNumber}`);
        const aircraft = this.aircrafts.get(flightNumber);
        if (aircraft) {
            aircraft.setAltitude(0);
        }
    }
}

// Example 3: Smart Home System
// Shows how Mediator can coordinate home automation

class SmartDevice {
    protected homeHub: Mediator;
    protected deviceName: string;
    protected isOn: boolean = false;

    constructor(hub: Mediator, name: string) {
        this.homeHub = hub;
        this.deviceName = name;
    }

    turnOn(): void {
        this.isOn = true;
        console.log(`${this.deviceName} turned ON`);
    }
}
```

```
        this.homeHub.sendMessage(`${this.deviceName} turned ON`,
this.deviceName);
    }

    turnOff(): void {
        this.isOn = false;
        console.log(`${this.deviceName} turned OFF`);
        this.homeHub.sendMessage(`${this.deviceName} turned OFF`,
this.deviceName);
    }

    getName(): string {
        return this.deviceName;
    }

    getStatus(): boolean {
        return this.isOn;
    }
}

class SmartLight extends SmartDevice {
    constructor(hub: Mediator, name: string) {
        super(hub, name);
    }
}

class SmartThermostat extends SmartDevice {
    private temperature: number = 70;

    constructor(hub: Mediator, name: string) {
        super(hub, name);
    }

    setTemperature(temp: number): void {
        this.temperature = temp;
        console.log(`${this.getName()} set to ${this.temperature}°F`);
    }

    getTemperature(): number {
        return this.temperature;
    }
}

class HomeHub implements Mediator {
    private devices: Map<string, SmartDevice> = new Map();

    addDevice(device: SmartDevice): void {
        this.devices.set(device.getName(), device);
    }

    sendMessage(message: string, sender: string): void {
        console.log(`Home Hub: ${message}`);

        // Example logic: if thermostat turns on, turn on lights
```

```
    if (sender.includes('Thermostat') && message.includes('turned ON')) {
      for (const [name, device] of this.devices) {
        if (name.includes('Light') && !device.getStatus()) {
          device.turnOn();
        }
      }
    }
  }
}

activateAwayMode(): void {
  console.log('Home Hub: Activating away mode...');
  for (const [name, device] of this.devices) {
    if (device.getStatus()) {
      device.turnOff();
    }
  }
}

activateHomeMode(): void {
  console.log('Home Hub: Activating home mode...');
  for (const [name, device] of this.devices) {
    if (name.includes('Light')) {
      device.turnOn();
    }
  }
}
}

// Demo code
console.log('=== Mediator Pattern Demo ===\n');

// Example 1: Chat Room
console.log('--- Chat Room Example ---');

const chatRoom = new ChatRoom();

const user1 = new User(chatRoom, 'Alice');
const user2 = new User(chatRoom, 'Bob');
const bot = new Bot(chatRoom, 'HelperBot');

chatRoom.addUser(user1);
chatRoom.addUser(user2);
chatRoom.addUser(bot);

console.log('\nAlice sends a message:');
user1.send('Hello everyone!');

console.log('\nBob sends a message:');
user2.send('Hi Alice!');

console.log('');

// Example 2: Air Traffic Control
console.log('--- Air Traffic Control Example ---');
```

```
const controlTower = new ControlTower();

const flight1 = new Aircraft(controlTower, 'AA123');
const flight2 = new Aircraft(controlTower, 'UA456');

controlTower.registerAircraft(flight1);
controlTower.registerAircraft(flight2);

console.log('\nFlight AA123 requests takeoff:');
controlTower.grantTakeoff('AA123');

console.log('\nFlight UA456 requests landing:');
controlTower.grantLanding('UA456');

console.log('');

// Example 3: Smart Home System
console.log('--- Smart Home System Example ---');

const homeHub = new HomeHub();

const livingRoomLight = new SmartLight(homeHub, 'Living Room Light');
const bedroomLight = new SmartLight(homeHub, 'Bedroom Light');
const thermostat = new SmartThermostat(homeHub, 'Main Thermostat');

homeHub.addDevice(livingRoomLight);
homeHub.addDevice(bedroomLight);
homeHub.addDevice(thermostat);

console.log('\nTurning on thermostat (should trigger lights:');
thermostat.turnOn();

console.log('\nActivating away mode:');
homeHub.activateAwayMode();

console.log('\nActivating home mode:');
homeHub.activateHomeMode();
```

Memento

Description: Captures and externalizes an object's **internal state** so it can be restored later, without violating encapsulation.

- **Why it is used:** To implement undo/redo functionality by saving snapshots of an object's state.
- **Where it is commonly used:** Text editors (undo history), game save states, transactional systems.

Code Example (JAVASCRIPT)

```
/*
 * MEMENTO DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Memento pattern lets you save and restore the previous state of an
object without revealing the details of its implementation.
 * It provides the ability to restore an object to its previous state
(undo).
 *
 * WHY DOES IT EXIST?
 * - When you need to implement undo/redo functionality
 * - When you want to create snapshots of an object's state
 * - When direct access to the object's internal representation would
violate encapsulation
 *
 * WHEN TO USE IT?
 * - When you need to save and restore the state of an object without
exposing its internal structure
 * - When you want to implement undo/redo operations
 * - When you need to maintain a history of states
 * - Common use cases: text editors (undo/redo), game states, database
transactions, configuration management
 *
 * PROS:
 * - Preserves encapsulation (doesn't expose internal state)
 * - Simplifies the originator (doesn't need to manage state history)
 * - Can provide snapshots at different times
 * - Supports undo/redo functionality
 *
 * CONS:
 * - Can be expensive if the originator has large state
 * - May consume significant memory if many mementos are stored
 * - Caretaker must track the originator's lifecycle
 * - Can be complex to implement for complex objects
 *
 * REAL-WORLD ANALOGY:
 * Think of a game save system. When you save a game, the game creates a
snapshot of your current state (position, health, inventory). This snapshot
(memento) is stored separately. Later, you can restore the game to that
exact state by loading the snapshot. The game doesn't need to know how the
snapshot is stored, just that it can be restored.
 */

// Memento - stores the internal state of the Originator
class TextEditorMemento {
  private content: string;
  private timestamp: string;

  constructor(content: string) {
    this.content = content;
    this.timestamp = this.getCurrentTime();
  }
}
```

```
    getContent(): string {
        return this.content;
    }

    getTimestamp(): string {
        return this.timestamp;
    }

    private getCurrentTime(): string {
        return new Date().toLocaleString();
    }
}

// Originator - creates a memento containing a snapshot of its current
// state
class TextEditor {
    private content: string = '';

    write(text: string): void {
        this.content += text;
        console.log(`Text: "${text}" added`);
    }

    setContent(newContent: string): void {
        this.content = newContent;
    }

    getContent(): string {
        return this.content;
    }

    save(): TextEditorMemento {
        console.log('Saving current state...');
        return new TextEditorMemento(this.content);
    }

    restore(memento: TextEditorMemento): void {
        this.content = memento.getContent();
        console.log(`Restored state from ${memento.getTimestamp()}`);
        console.log(`Current content: "${this.content}"`);
    }
}

// Caretaker - manages mementos, never operates on their contents
class History {
    private mementos: TextEditorMemento[] = [];

    push(memento: TextEditorMemento): void {
        this.mementos.push(memento);
    }

    pop(): TextEditorMemento | null {
        if (this.mementos.length === 0) {

```

```
        return null;
    }
    return this.mementos.pop()!;
}

size(): number {
    return this.mementos.length;
}
}

// Example 2: Game State Memento
// Shows how Memento can be used for game saves

class GameStateMemento {
    private level: number;
    private health: number;
    private score: number;
    private position: string;

    constructor(level: number, health: number, score: number, position:
string) {
        this.level = level;
        this.health = health;
        this.score = score;
        this.position = position;
    }

    getLevel(): number { return this.level; }
    getHealth(): number { return this.health; }
    getScore(): number { return this.score; }
    getPosition(): string { return this.position; }
}

class GameCharacter {
    private level: number = 1;
    private health: number = 100;
    private score: number = 0;
    private position: string = 'Start';

    moveTo(newPosition: string): void {
        this.position = newPosition;
        console.log(`Moved to ${this.position}`);
    }

    takeDamage(damage: number): void {
        this.health -= damage;
        if (this.health < 0) this.health = 0;
        console.log(`Took ${damage} damage. Health: ${this.health}`);
    }

    gainScore(points: number): void {
        this.score += points;
        console.log(`Gained ${points} points. Score: ${this.score}`);
    }
}
```

```
levelUp(): void {
    this.level++;
    this.health = 100;
    console.log(`Level up! Now at level ${this.level}`);
}

displayStatus(): void {
    console.log('=== Character Status ===');
    console.log(`Level: ${this.level}`);
    console.log(`Health: ${this.health}`);
    console.log(`Score: ${this.score}`);
    console.log(`Position: ${this.position}`);
    console.log('=====');
}

saveState(): GameStateMemento {
    console.log('Saving game state...');
    return new GameStateMemento(this.level, this.health, this.score,
this.position);
}

restoreState(memento: GameStateMemento): void {
    this.level = memento.getLevel();
    this.health = memento.getHealth();
    this.score = memento.getScore();
    this.position = memento.getPosition();
    console.log('Game state restored!');
    this.displayStatus();
}
}

class GameSaveManager {
    private saves: GameStateMemento[] = [];

    createSave(save: GameStateMemento): void {
        this.saves.push(save);
        console.log(`Save created. Total saves: ${this.saves.length}`);
    }

    loadSave(index: number): GameStateMemento | null {
        if (index >= 0 && index < this.saves.length) {
            console.log(`Loading save ${index + 1}...`);
            return this.saves[index];
        }
        return null;
    }

    getSaveCount(): number {
        return this.saves.length;
    }
}

// Example 3: Configuration Memento
```

```
// Shows how Memento can be used for configuration management

class ConfigurationMemento {
  private theme: string;
  private fontSize: number;
  private notificationsEnabled: boolean;

  constructor(theme: string, fontSize: number, notifications: boolean) {
    this.theme = theme;
    this.fontSize = fontSize;
    this.notificationsEnabled = notifications;
  }

  getTheme(): string { return this.theme; }
  getFontSize(): number { return this.fontSize; }
  getNotificationsEnabled(): boolean { return this.notificationsEnabled; }
}

class ApplicationSettings {
  private theme: string = 'light';
  private fontSize: number = 12;
  private notificationsEnabled: boolean = true;

  setTheme(newTheme: string): void {
    this.theme = newTheme;
    console.log(`Theme changed to ${this.theme}`);
  }

  setFontSize(size: number): void {
    this.fontSize = size;
    console.log(`Font size set to ${this.fontSize}`);
  }

  setNotificationsEnabled(enabled: boolean): void {
    this.notificationsEnabled = enabled;
    console.log(`Notifications ${enabled ? 'enabled' : 'disabled'}`);
  }

  displaySettings(): void {
    console.log('=== Current Settings ===');
    console.log(`Theme: ${this.theme}`);
    console.log(`Font Size: ${this.fontSize}`);
    console.log(`Notifications: ${this.notificationsEnabled ? 'Enabled' : 'Disabled'}`);
    console.log('=====');
  }

  saveConfiguration(): ConfigurationMemento {
    return new ConfigurationMemento(this.theme, this.fontSize,
this.notificationsEnabled);
  }

  restoreConfiguration(memento: ConfigurationMemento): void {
    this.theme = memento.getTheme();
  }
}
```

```
        this.fontSize = memento.getFontSize();
        this.notificationsEnabled = memento.getNotificationsEnabled();
        console.log('Configuration restored!');
        this.displaySettings();
    }
}

// Demo code
console.log('=== Memento Pattern Demo ===\n');

// Example 1: Text Editor with Undo
console.log('--- Text Editor with Undo ---');

const editor = new TextEditor();
const history = new History();

editor.write('Hello ');
history.push(editor.save());

editor.write('World ');
history.push(editor.save());

editor.write('!');
history.push(editor.save());

console.log(`\nCurrent content: "${editor.getContent()}"`);
console.log(`History size: ${history.size()}`);

console.log('\nPerforming undo...');
const memento1 = history.pop();
if (memento1) {
    editor.restore(memento1);
}

console.log('\nPerforming undo...');
const memento2 = history.pop();
if (memento2) {
    editor.restore(memento2);
}

console.log('');

// Example 2: Game Save System
console.log('--- Game Save System ---');

const player = new GameCharacter();
const saveManager = new GameSaveManager();

console.log('\nInitial state:');
player.displayStatus();

console.log('\nPlaying game...');
player.moveTo('Forest');
player.gainScore(100);
```

```
player.takeDamage(10);

console.log('\nCreating save 1...');
saveManager.createSave(player.saveState());

console.log('\nContinuing game...');
player.moveTo('Castle');
player.levelUp();
player.gainScore(500);
player.takeDamage(30);

console.log('\nCreating save 2...');
saveManager.createSave(player.saveState());

console.log('\nCurrent state:');
player.displayStatus();

console.log('\nLoading save 1...');
const save1 = saveManager.loadSave(0);
if (save1) {
    player.restoreState(save1);
}

console.log('');

// Example 3: Configuration Management
console.log('--- Configuration Management Example ---');

const settings = new ApplicationSettings();

console.log('\nDefault settings:');
settings.displaySettings();

const originalConfig = settings.saveConfiguration();

console.log('\nChanging settings...');
settings.setTheme('dark');
settings.setFontSize(14);
settings.setNotificationsEnabled(false);

console.log('\nNew settings:');
settings.displaySettings();

console.log('\nRestoring original settings...');
settings.restoreConfiguration(originalConfig);
```

Observer

Description: Defines a **one-to-many dependency** between objects so that when one object changes state, all its dependents are notified and updated automatically.

- **Why it is used:** To implement event-driven systems where multiple parts of an application need to react to changes in a shared object.
- **Where it is commonly used:** Event listeners, pub/sub systems, real-time dashboards, MVC architecture (model notifying views).

Code Example (JAVASCRIPT)

```

/*
 * OBSERVER DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Observer pattern defines a subscription mechanism to notify multiple
objects about any events that happen to the object they are observing.
 * It establishes a one-to-many dependency between objects so that when one
object changes state, all its dependents are notified.
 *
 * WHY DOES IT EXIST?
 * - When you need to notify multiple objects about changes to a single
object
 * - When you want to decouple the subject from its observers
 * - When you want to broadcast changes to multiple subscribers dynamically
 *
 * WHEN TO USE IT?
 * - When changes to one object require changing other objects
 * - When the set of objects that need to be notified is unknown or dynamic
 * - When you want to implement a publish-subscribe model
 * - Common use cases: GUI event handling, news feeds, social media
notifications, stock price updates
 *
 * PROS:
 * - Open/Closed Principle (can add new observers without changing the
subject)
 * - Establishes loose coupling between subject and observers
 * - Can broadcast communications to multiple objects
 * - Supports dynamic subscription/unsubscription
 *
 * CONS:
 * - Can lead to unexpected updates if observers are not careful
 * - Can cause performance issues with many observers
 * - Debugging can be difficult due to indirect calls
 * - May cause memory leaks if observers don't unsubscribe
 *
 * REAL-WORLD ANALOGY:
 * Think of a YouTube channel (subject) and subscribers (observers). When
the channel uploads a new video, all subscribers get notified. Subscribers
can subscribe or unsubscribe at any time. The channel doesn't need to know
who is subscribed - it just broadcasts notifications to all current
subscribers.
 */

// Observer interface - declares the update method

```

```
interface Observer {
  update(message: string): void;
}

// Subject interface - declares methods for attaching, detaching, and
// notifying observers
abstract class Subject {
  protected observers: Observer[] = [];

  attach(observer: Observer): void {
    this.observers.push(observer);
    console.log('Attached an observer');
  }

  detach(observer: Observer): void {
    const index = this.observers.indexOf(observer);
    if (index !== -1) {
      this.observers.splice(index, 1);
      console.log('Detached an observer');
    }
  }

  notify(message: string): void {
    console.log('Notifying observers...');
    for (const observer of this.observers) {
      observer.update(message);
    }
  }
}

// Concrete Subject
class YouTubeChannel extends Subject {
  private channelName: string;

  constructor(name: string) {
    super();
    this.channelName = name;
  }

  uploadVideo(videoTitle: string): void {
    console.log(`\n${this.channelName} uploaded: ${videoTitle}`);
    this.notify(`New video: ${videoTitle}`);
  }

  goLive(streamTitle: string): void {
    console.log(`\n${this.channelName} is live: ${streamTitle}`);
    this.notify(`Live stream started: ${streamTitle}`);
  }
}

// Concrete Observer 1
class Subscriber implements Observer {
  private name: string;
```

```
    constructor(name: string) {
        this.name = name;
    }

    update(message: string): void {
        console.log(` ${this.name} received notification: ${message}`);
    }
}

// Concrete Observer 2
class NotificationService implements Observer {
    update(message: string): void {
        console.log(` [NotificationService] Push notification sent:
${message}`);
    }
}

// Example 2: Stock Market
// Shows how Observer can be used for real-time updates

interface StockObserver {
    onStockPriceChange(symbol: string, price: number): void;
}

class StockMarket {
    private observers: StockObserver[] = [];
    private stockPrices: Map<string, number> = new Map();

    addObserver(observer: StockObserver): void {
        this.observers.push(observer);
    }

    removeObserver(observer: StockObserver): void {
        const index = this.observers.indexOf(observer);
        if (index !== -1) {
            this.observers.splice(index, 1);
        }
    }

    updateStockPrice(symbol: string, price: number): void {
        this.stockPrices.set(symbol, price);
        console.log(`\n${symbol} price updated to ${price}`);
        this.notifyObservers(symbol, price);
    }

    private notifyObservers(symbol: string, price: number): void {
        for (const observer of this.observers) {
            observer.onStockPriceChange(symbol, price);
        }
    }
}

class Investor implements StockObserver {
    private name: string;
```

```

private portfolio: Map<string, number> = new Map();

constructor(name: string) {
  this.name = name;
}

buyStock(symbol: string, shares: number): void {
  const current = this.portfolio.get(symbol) || 0;
  this.portfolio.set(symbol, current + shares);
  console.log(`${this.name} bought ${shares} shares of ${symbol}`);
}

onStockPriceChange(symbol: string, price: number): void {
  const shares = this.portfolio.get(symbol);
  if (shares !== undefined && shares > 0) {
    console.log(`  ${this.name} owns ${shares} shares of ${symbol}
(current price: ${price})`);
  }
}
}

class TradingBot implements StockObserver {
  onStockPriceChange(symbol: string, price: number): void {
    console.log(`  [TradingBot] Analyzing ${symbol} at ${price}`);
    if (price > 100) {
      console.log('  [TradingBot] Recommendation: SELL');
    } else if (price < 50) {
      console.log('  [TradingBot] Recommendation: BUY');
    }
  }
}

// Example 3: Weather Station
// Shows how Observer can be used for sensor data

interface WeatherObserver {
  onWeatherUpdate(temperature: number, humidity: number, pressure: number):
void;
}

class WeatherStation {
  private observers: WeatherObserver[] = [];
  private temperature: number = 0;
  private humidity: number = 0;
  private pressure: number = 0;

  addObserver(observer: WeatherObserver): void {
    this.observers.push(observer);
  }

  removeObserver(observer: WeatherObserver): void {
    const index = this.observers.indexOf(observer);
    if (index !== -1) {
      this.observers.splice(index, 1);
    }
  }
}

```

```
    }
  }

  setMeasurements(temp: number, hum: number, press: number): void {
    this.temperature = temp;
    this.humidity = hum;
    this.pressure = press;
    console.log(`\nWeather updated: ${this.temperature}°C,
    ${this.humidity}% humidity, ${this.pressure} hPa`);
    this.notifyObservers();
  }

  private notifyObservers(): void {
    for (const observer of this.observers) {
      observer.onWeatherUpdate(this.temperature, this.humidity,
    this.pressure);
    }
  }
}

class DisplayDevice implements WeatherObserver {
  private deviceName: string;

  constructor(name: string) {
    this.deviceName = name;
  }

  onWeatherUpdate(temperature: number, humidity: number, pressure: number):
  void {
    console.log(` [${this.deviceName}] Displaying: ${temperature}°C,
    ${humidity}%, ${pressure} hPa`);
  }
}

class AlertSystem implements WeatherObserver {
  onWeatherUpdate(temperature: number, humidity: number, pressure: number):
  void {
    console.log(' [AlertSystem] Checking conditions...');
    if (temperature > 35) {
      console.log(' [AlertSystem] WARNING: High temperature!');
    }
    if (humidity > 80) {
      console.log(' [AlertSystem] WARNING: High humidity!');
    }
    if (pressure < 980) {
      console.log(' [AlertSystem] WARNING: Low pressure (storm coming)!');
    }
  }
}

// Demo code
console.log('=== Observer Pattern Demo ===\n');

// Example 1: YouTube Channel
```

```
console.log('--- YouTube Channel Example ---');

const channel = new YouTubeChannel('Tech Channel');

const subscriber1 = new Subscriber('Alice');
const subscriber2 = new Subscriber('Bob');
const notificationService = new NotificationService();

channel.attach(subscriber1);
channel.attach(subscriber2);
channel.attach(notificationService);

channel.uploadVideo('Design Patterns Tutorial');

channel.detach(subscriber2);

channel.goLive('Live Q&A Session');

console.log('');

// Example 2: Stock Market
console.log('--- Stock Market Example ---');

const market = new StockMarket();

const investor1 = new Investor('John');
const investor2 = new Investor('Jane');
const tradingBot = new TradingBot();

market.addObserver(investor1);
market.addObserver(investor2);
market.addObserver(tradingBot);

investor1.buyStock('AAPL', 100);
investor2.buyStock('GOOGL', 50);

market.updateStockPrice('AAPL', 150.50);
market.updateStockPrice('GOOGL', 45.00);
market.updateStockPrice('TSLA', 200.00);

console.log('');

// Example 3: Weather Station
console.log('--- Weather Station Example ---');

const station = new WeatherStation();

const phoneDisplay = new DisplayDevice('Phone');
const watchDisplay = new DisplayDevice('Smart Watch');
const alertSystem = new AlertSystem();

station.addObserver(phoneDisplay);
station.addObserver(watchDisplay);
station.addObserver(alertSystem);
```

```
station.setMeasurements(25.0, 60.0, 1013.0);  
station.setMeasurements(38.0, 85.0, 975.0);  
station.setMeasurements(20.0, 50.0, 1020.0);
```

State

Description: Allows an object to **alter its behavior when its internal state changes**. The object will appear to change its class.

- **Why it is used:** To replace complex conditional statements (`if/switch`) that depend on the object's state with dedicated state classes.
- **Where it is commonly used:** Traffic light systems, vending machines, order status workflows, game character states.

Code Example (JAVASCRIPT)

```
/*  
 * STATE DESIGN PATTERN  
 *  
 * WHAT IS IT?  
 * The State pattern allows an object to alter its behavior when its  
internal state changes.  
 * The object appears to change its class by delegating state-specific  
behavior to different state objects.  
 *  
 * WHY DOES IT EXIST?  
 * - When an object's behavior depends on its state and must change at  
runtime  
 * - When you have large conditional statements that switch between states  
 * - When you want to avoid monolithic conditional logic  
 *  
 * WHEN TO USE IT?  
 * - When an object has many states and its behavior changes based on those  
states  
 * - When you have large conditional operators that switch between states  
 * - When you want to eliminate conditional logic from the main class  
 * - Common use cases: game characters, vending machines, audio/video  
players, document editors  
 *  
 * PROS:  
 * - Single Responsibility Principle (each state is a separate class)  
 * - Open/Closed Principle (can add new states without changing existing  
code)  
 * - Eliminates conditional logic from the context  
 * - Makes state transitions explicit  
 *  
 * CONS:  
 * - Can increase the number of classes in the system  
 * - State transitions can be complex to manage  
 */
```

```
* - May make the code more complex for simple scenarios
* - Can lead to tight coupling between states
*
* REAL-WORLD ANALOGY:
* Think of a vending machine. It has different states: "No Coin", "Has
Coin", "Dispensing", "Out of Stock". When you insert a coin, it transitions
from "No Coin" to "Has Coin". When you press a button, it transitions to
"Dispensing". The machine's behavior changes based on its current state,
but the machine itself doesn't change - it just delegates to the
appropriate state object.
*/

// State interface - declares the state-specific methods
interface State {
  insertCoin(): void;
  ejectCoin(): void;
  pressButton(): void;
  dispense(): void;
}

// Context - maintains a reference to the current state
class VendingMachine {
  private currentState: State;
  private coinCount: number = 0;
  private stockCount: number;

  constructor(initialStock: number) {
    this.stockCount = initialStock;
    // Initial state depends on stock
    if (initialStock > 0) {
      this.currentState = new NoCoinState(this);
    } else {
      this.currentState = new SoldOutState(this);
    }
  }

  setState(state: State): void {
    this.currentState = state;
  }

  insertCoin(): void {
    this.currentState.insertCoin();
  }

  ejectCoin(): void {
    this.currentState.ejectCoin();
  }

  pressButton(): void {
    this.currentState.pressButton();
  }

  dispense(): void {
    this.currentState.dispense();
  }
}
```

```
}

addCoin(): void {
  this.coinCount++;
  console.log(`Coin inserted. Total coins: ${this.coinCount}`);
}

returnCoin(): void {
  if (this.coinCount > 0) {
    this.coinCount--;
    console.log(`Coin returned. Total coins: ${this.coinCount}`);
  }
}

releaseProduct(): void {
  if (this.stockCount > 0) {
    this.stockCount--;
    this.coinCount--;
    console.log(`Product dispensed. Stock: ${this.stockCount}, Coins:
${this.coinCount}`);
  }
}

getCoinCount(): number {
  return this.coinCount;
}

getStockCount(): number {
  return this.stockCount;
}
}

// Concrete State 1: No Coin
class NoCoinState implements State {
  private machine: VendingMachine;

  constructor(machine: VendingMachine) {
    this.machine = machine;
  }

  insertCoin(): void {
    console.log('Coin inserted...');
    this.machine.addCoin();
    this.machine.setState(new HasCoinState(this.machine));
  }

  ejectCoin(): void {
    console.log('No coin to eject');
  }

  pressButton(): void {
    console.log('Please insert a coin first');
  }
}
```

```
dispense(): void {
    console.log('Please insert a coin first');
}

// Concrete State 2: Has Coin
class HasCoinState implements State {
    private machine: VendingMachine;

    constructor(machine: VendingMachine) {
        this.machine = machine;
    }

    insertCoin(): void {
        console.log('Coin already inserted');
    }

    ejectCoin(): void {
        console.log('Coin ejected');
        this.machine.returnCoin();
        this.machine.setState(new NoCoinState(this.machine));
    }

    pressButton(): void {
        console.log('Button pressed...');
        if (this.machine.getStockCount() > 0) {
            this.machine.setState(new DispensingState(this.machine));
            this.machine.dispense();
        } else {
            console.log('Out of stock');
            this.machine.setState(new SoldOutState(this.machine));
        }
    }

    dispense(): void {
        console.log('Press button to dispense');
    }
}

// Concrete State 3: Dispensing
class DispensingState implements State {
    private machine: VendingMachine;

    constructor(machine: VendingMachine) {
        this.machine = machine;
    }

    insertCoin(): void {
        console.log('Please wait, dispensing...');
    }

    ejectCoin(): void {
        console.log('Cannot eject during dispensing');
    }
}
```

```
    pressButton(): void {
        console.log('Already dispensing...');
    }

    dispense(): void {
        this.machine.releaseProduct();
        if (this.machine.getStockCount() > 0) {
            this.machine.setState(new NoCoinState(this.machine));
        } else {
            this.machine.setState(new SoldOutState(this.machine));
        }
    }
}

// Concrete State 4: Sold Out
class SoldOutState implements State {
    private machine: VendingMachine;

    constructor(machine: VendingMachine) {
        this.machine = machine;
    }

    insertCoin(): void {
        console.log('Cannot insert coin, machine is sold out');
    }

    ejectCoin(): void {
        console.log('Cannot eject, no coin inserted');
    }

    pressButton(): void {
        console.log('Machine is sold out');
    }

    dispense(): void {
        console.log('Machine is sold out');
    }
}

// Example 2: Audio Player States
// Shows how State can be used for media players

interface AudioPlayerState {
    clickPlay(): void;
    clickStop(): void;
    clickNext(): void;
    clickPrevious(): void;
}

class AudioPlayer {
    private currentState: AudioPlayerState;
    private isPlaying: boolean = false;
    private currentTrack: number = 1;
```

```
private totalTracks: number;

constructor(tracks: number) {
  this.totalTracks = tracks;
  this.currentState = new StoppedState(this);
}

setState(state: AudioPlayerState): void {
  this.currentState = state;
}

clickPlay(): void {
  this.currentState.clickPlay();
}

clickStop(): void {
  this.currentState.clickStop();
}

clickNext(): void {
  this.currentState.clickNext();
}

clickPrevious(): void {
  this.currentState.clickPrevious();
}

startPlaying(): void {
  this.isPlaying = true;
  console.log(`Started playing track ${this.currentTrack}`);
}

stopPlaying(): void {
  this.isPlaying = false;
  console.log('Stopped playing');
}

nextTrack(): void {
  this.currentTrack = (this.currentTrack % this.totalTracks) + 1;
  console.log(`Moved to track ${this.currentTrack}`);
}

previousTrack(): void {
  this.currentTrack = this.currentTrack === 1 ? this.totalTracks :
this.currentTrack - 1;
  console.log(`Moved to track ${this.currentTrack}`);
}

getIsPlaying(): boolean {
  return this.isPlaying;
}
}

class PlayingState implements AudioPlayerState {
```

```
private player: AudioPlayer;

constructor(player: AudioPlayer) {
  this.player = player;
}

clickPlay(): void {
  console.log('Already playing');
}

clickStop(): void {
  console.log('Stopping playback...');
  this.player.stopPlaying();
  this.player.setState(new StoppedState(this.player));
}

clickNext(): void {
  console.log('Skipping to next track...');
  this.player.nextTrack();
}

clickPrevious(): void {
  console.log('Going to previous track...');
  this.player.previousTrack();
}
}

class StoppedState implements AudioPlayerState {
  private player: AudioPlayer;

  constructor(player: AudioPlayer) {
    this.player = player;
  }

  clickPlay(): void {
    console.log('Starting playback...');
    this.player.startPlaying();
    this.player.setState(new PlayingState(this.player));
  }

  clickStop(): void {
    console.log('Already stopped');
  }

  clickNext(): void {
    console.log('Skipping to next track...');
    this.player.nextTrack();
  }

  clickPrevious(): void {
    console.log('Going to previous track...');
    this.player.previousTrack();
  }
}
```

```
// Demo code
console.log('=== State Pattern Demo ===\n');

// Example 1: Vending Machine
console.log('--- Vending Machine Example ---');

const machine = new VendingMachine(5); // 5 items in stock

console.log('\nAttempt to press button without coin:');
machine.pressButton();

console.log('\nInsert coin:');
machine.insertCoin();

console.log('\nPress button:');
machine.pressButton();

console.log('\nInsert another coin:');
machine.insertCoin();

console.log('\nPress button:');
machine.pressButton();

console.log('\nEject coin:');
machine.ejectCoin();

console.log('');

// Example 2: Audio Player
console.log('--- Audio Player Example ---');

const player = new AudioPlayer(10); // 10 tracks

console.log('\nClick play (stopped state):');
player.clickPlay();

console.log('\nClick next:');
player.clickNext();

console.log('\nClick stop:');
player.clickStop();

console.log('\nClick previous:');
player.clickPrevious();

console.log('\nClick play again:');
player.clickPlay();
```

Strategy

Description: Defines a **family of algorithms**, encapsulates each one, and makes them interchangeable. The strategy lets the algorithm vary independently from clients that use it.

- **Why it is used:** To select behavior at runtime without changing the client that uses it. Promotes the Open/Closed Principle.
- **Where it is commonly used:** Sorting algorithms (choose bubble sort vs. quicksort at runtime), payment processing systems, data compression strategies.

Code Example (JAVASCRIPT)

```
/*
 * STRATEGY DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Strategy pattern defines a family of algorithms, encapsulates each
one, and makes them interchangeable.
 * It lets the algorithm vary independently from clients that use it.
 *
 * WHY DOES IT EXIST?
 * - When you want to define a family of algorithms and make them
interchangeable
 * - When you want to avoid conditional statements for selecting algorithms
 * - When you want to separate algorithm implementation from the code that
uses it
 *
 * WHEN TO USE IT?
 * - When you have multiple ways to do an operation and want to switch
between them at runtime
 * - When you want to isolate the algorithm implementation from the client
code
 * - When you have complex conditional logic that selects different
algorithms
 * - Common use cases: sorting algorithms, payment processing, compression,
route planning, validation
 *
 * PROS:
 * - Open/Closed Principle (can add new strategies without changing
existing code)
 * - Separates algorithm implementation from the code that uses it
 * - Eliminates conditional statements for selecting algorithms
 * - Allows runtime selection of algorithms
 *
 * CONS:
 * - Clients must be aware of the differences between strategies
 * - Can increase the number of classes in the system
 * - May lead to performance overhead if strategies are complex
 * - Can make the code more complex for simple scenarios
 *
 * REAL-WORLD ANALOGY:
 * Think of a GPS navigation app. It can use different route calculation
strategies: fastest route, shortest distance, avoid highways, scenic route.
```

You can switch between these strategies at runtime without changing the app itself. The app (context) delegates the route calculation to the selected strategy.

```
*/

// Strategy interface - declares the algorithm method
interface PaymentStrategy {
  pay(amount: number): void;
}

// Concrete Strategy 1: Credit Card
class CreditCardPayment implements PaymentStrategy {
  private cardNumber: string;
  private expiryDate: string;
  private cvv: string;

  constructor(card: string, expiry: string, cvv: string) {
    this.cardNumber = card;
    this.expiryDate = expiry;
    this.cvv = cvv;
  }

  pay(amount: number): void {
    console.log(`Paying ${amount} with Credit Card ending in
    ${this.cardNumber.slice(-4)}`);
    console.log(`  Card: ****-****-****-${this.cardNumber.slice(-4)}`);
    console.log(`  Expiry: ${this.expiryDate}`);
  }
}

// Concrete Strategy 2: PayPal
class PayPalPayment implements PaymentStrategy {
  private email: string;
  private password: string;

  constructor(email: string, password: string) {
    this.email = email;
    this.password = password;
  }

  pay(amount: number): void {
    console.log(`Paying ${amount} with PayPal`);
    console.log(`  Account: ${this.email}`);
  }
}

// Concrete Strategy 3: Crypto
class CryptoPayment implements PaymentStrategy {
  private walletAddress: string;
  private privateKey: string;

  constructor(wallet: string, key: string) {
    this.walletAddress = wallet;
    this.privateKey = key;
  }
}
```

```

    }

    pay(amount: number): void {
        console.log(`Paying ${amount} with Cryptocurrency`);
        console.log(`  Wallet: ${this.walletAddress.slice(0, 10)}...`);
    }
}

// Context - uses the strategy
class ShoppingCart {
    private paymentStrategy: PaymentStrategy | null = null;
    private items: { name: string; price: number }[] = [];

    setPaymentStrategy(strategy: PaymentStrategy): void {
        this.paymentStrategy = strategy;
    }

    addItem(item: string, price: number): void {
        this.items.push({ name: item, price: price });
    }

    getTotal(): number {
        return this.items.reduce((total, item) => total + item.price, 0);
    }

    checkout(): void {
        console.log('\n=== Checkout ===');
        console.log('Items:');
        for (const item of this.items) {
            console.log(`  - ${item.name}: ${item.price}`);
        }
        console.log(`Total: ${this.getTotal()}`);

        if (this.paymentStrategy) {
            this.paymentStrategy.pay(this.getTotal());
        } else {
            console.log('No payment method selected');
        }
    }
}

// Example 2: Sorting Strategies
// Shows how Strategy can be used for different algorithms

interface SortingStrategy {
    sort(data: number[]): void;
}

class BubbleSort implements SortingStrategy {
    sort(data: number[]): void {
        console.log('Using Bubble Sort...');
        const n = data.length;
        for (let i = 0; i < n - 1; i++) {
            for (let j = 0; j < n - i - 1; j++) {

```

```
        if (data[j] > data[j + 1]) {
            [data[j], data[j + 1]] = [data[j + 1], data[j]];
        }
    }
}

class QuickSort implements SortingStrategy {
    sort(data: number[]): void {
        console.log('Using Quick Sort...');
        this.quickSort(data, 0, data.length - 1);
    }

    private quickSort(data: number[], low: number, high: number): void {
        if (low < high) {
            const pi = this.partition(data, low, high);
            this.quickSort(data, low, pi - 1);
            this.quickSort(data, pi + 1, high);
        }
    }

    private partition(data: number[], low: number, high: number): number {
        const pivot = data[high];
        let i = low - 1;
        for (let j = low; j < high; j++) {
            if (data[j] < pivot) {
                i++;
                [data[i], data[j]] = [data[j], data[i]];
            }
        }
        [data[i + 1], data[high]] = [data[high], data[i + 1]];
        return i + 1;
    }
}

class MergeSort implements SortingStrategy {
    sort(data: number[]): void {
        console.log('Using Merge Sort...');
        this.mergeSort(data, 0, data.length - 1);
    }

    private mergeSort(data: number[], left: number, right: number): void {
        if (left < right) {
            const mid = left + Math.floor((right - left) / 2);
            this.mergeSort(data, left, mid);
            this.mergeSort(data, mid + 1, right);
            this.merge(data, left, mid, right);
        }
    }

    private merge(data: number[], left: number, mid: number, right: number):
void {
        const n1 = mid - left + 1;
```

```
    const n2 = right - mid;

    const L = data.slice(left, left + n1);
    const R = data.slice(mid + 1, mid + 1 + n2);

    let i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            data[k++] = L[i++];
        } else {
            data[k++] = R[j++];
        }
    }

    while (i < n1) data[k++] = L[i++];
    while (j < n2) data[k++] = R[j++];
}

class Sorter {
    private strategy: SortingStrategy | null = null;

    setSortingStrategy(strategy: SortingStrategy): void {
        this.strategy = strategy;
    }

    sortData(data: number[]): void {
        console.log('\nOriginal data: ' + data.join(' '));

        if (this.strategy) {
            this.strategy.sort(data);
        } else {
            console.log('No sorting strategy selected');
        }

        console.log('Sorted data: ' + data.join(' '));
    }
}

// Example 3: Route Planning Strategies
// Shows how Strategy can be used for navigation

interface RouteStrategy {
    calculateRoute(from: string, to: string): void;
}

class FastestRoute implements RouteStrategy {
    calculateRoute(from: string, to: string): void {
        console.log(`Calculating fastest route from ${from} to ${to}`);
        console.log('    Using highways and main roads to minimize time');
        console.log('    Estimated time: 25 minutes');
    }
}
```

```
class ShortestRoute implements RouteStrategy {
  calculateRoute(from: string, to: string): void {
    console.log(`Calculating shortest route from ${from} to ${to}`);
    console.log('  Using direct path to minimize distance');
    console.log('  Estimated distance: 15 km');
  }
}

class ScenicRoute implements RouteStrategy {
  calculateRoute(from: string, to: string): void {
    console.log(`Calculating scenic route from ${from} to ${to}`);
    console.log('  Using parks and landmarks for best views');
    console.log('  Estimated time: 45 minutes');
  }
}

class NavigationSystem {
  private strategy: RouteStrategy | null = null;

  setRouteStrategy(strategy: RouteStrategy): void {
    this.strategy = strategy;
  }

  navigate(from: string, to: string): void {
    console.log('\n=== Navigation ===');
    console.log(`From: ${from}`);
    console.log(`To: ${to}`);

    if (this.strategy) {
      this.strategy.calculateRoute(from, to);
    } else {
      console.log('No route strategy selected');
    }
  }
}

// Demo code
console.log('=== Strategy Pattern Demo ===\n');

// Example 1: Payment Processing
console.log('--- Payment Processing Example ---');

const cart = new ShoppingCart();
cart.addItem('Laptop', 999.99);
cart.addItem('Mouse', 29.99);
cart.addItem('Keyboard', 79.99);

console.log('\nPaying with Credit Card:');
cart.setPaymentStrategy(new CreditCardPayment('4111111111111111', '12/25', '123'));
cart.checkout();

console.log('\nPaying with PayPal:');
cart.setPaymentStrategy(new PayPalPayment('user@example.com', 'password'));
```

```
cart.checkout();

console.log('\nPaying with Crypto:');
cart.setPaymentStrategy(new CryptoPayment('0x1234567890abcdef',
'private_key'));
cart.checkout();

// Example 2: Sorting Algorithms
console.log('\n--- Sorting Algorithms Example ---');

const sorter = new Sorter();
const data = [64, 34, 25, 12, 22, 11, 90];

sorter.setSortingStrategy(new BubbleSort());
sorter.sortData([...data]);

sorter.setSortingStrategy(new QuickSort());
sorter.sortData([...data]);

sorter.setSortingStrategy(new MergeSort());
sorter.sortData([...data]);

// Example 3: Route Planning
console.log('\n--- Route Planning Example ---');

const nav = new NavigationSystem();

console.log('\nFastest route:');
nav.setRouteStrategy(new FastestRoute());
nav.navigate('Home', 'Office');

console.log('\nShortest route:');
nav.setRouteStrategy(new ShortestRoute());
nav.navigate('Home', 'Office');

console.log('\nScenic route:');
nav.setRouteStrategy(new ScenicRoute());
nav.navigate('Home', 'Office');
```

Template Method

Description: Defines the **skeleton of an algorithm** in a base class, but lets subclasses override specific steps without changing the algorithm's overall structure.

- **Why it is used:** To avoid code duplication when multiple classes share the same algorithm structure but differ in specific steps.
- **Where it is commonly used:** Data parsing pipelines, report generation systems, game AI turn sequences.

Code Example (JAVASCRIPT)

```
/*
 * TEMPLATE METHOD DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Template Method pattern defines the skeleton of an algorithm in the
base class but lets subclasses override specific steps without changing the
algorithm's structure.
 * It's a behavioral pattern that uses inheritance to define parts of an
algorithm.
 *
 * WHY DOES IT EXIST?
 * - When you have an algorithm with invariant parts and variable parts
 * - When you want to avoid code duplication by extracting common behavior
 * - When you want to control the extension points of an algorithm
 *
 * WHEN TO USE IT?
 * - When you have several classes that contain almost identical algorithms
with minor differences
 * - When you want to let subclasses extend only certain parts of a large
algorithm
 * - When you want to enforce the overall structure of an algorithm while
allowing customization
 * - Common use cases: data processing pipelines, frameworks, build
systems, report generation
 *
 * PROS:
 * - Reuses common code in the base class
 * - Follows Open/Closed Principle (extend behavior without modifying
existing code)
 * - Provides control over the extension points
 * - Reduces code duplication
 *
 * CONS:
 * - Can be difficult to maintain if the template method is complex
 * - May limit flexibility due to rigid structure
 * - Can lead to class explosion if many variations are needed
 * - Subclasses may need to implement methods they don't use
 *
 * REAL-WORLD ANALOGY:
 * Think of cooking a recipe. The recipe (template method) defines the
steps: prepare ingredients, cook, serve. The specific ingredients and
cooking methods can vary (subclass implementations), but the overall
structure remains the same. You can make pasta, curry, or stir-fry using
the same recipe structure with different implementations.
 */

// Abstract Class - defines the template method and abstract primitive
operations
abstract class DataProcessor {
  // Template method - defines the algorithm structure
  processData(): void {
    console.log('\n=== Starting Data Processing ===');
```

```
        this.readData();
        this.validateData();
        this.transformData();
        this.saveData();
        console.log('=== Data Processing Complete ===');
    }

    // Primitive operations - to be implemented by subclasses
    protected abstract readData(): void;
    protected abstract validateData(): void;
    protected abstract transformData(): void;
    protected abstract saveData(): void;

    // Hook method - optional override point
    protected logProcessing(): void {
        console.log('Processing logged');
    }
}

// Concrete Class 1: CSV Data Processor
class CSVDataProcessor extends DataProcessor {
    private data: string = '';

    protected readData(): void {
        console.log('Reading CSV file...');
        this.data = 'name,age,city\nJohn,30,NYC\nJane,25,LA';
    }

    protected validateData(): void {
        console.log('Validating CSV format...');
        console.log('CSV format is valid');
    }

    protected transformData(): void {
        console.log('Transforming CSV to JSON...');
        // Simulate transformation
        this.data = ' [{"name":"John","age":30,"city":"NYC"}, {"name":"Jane","age":25,"city":"LA"} ]';
    }

    protected saveData(): void {
        console.log('Saving JSON file...');
        console.log(`Data saved: ${this.data}`);
    }
}

// Concrete Class 2: XML Data Processor
class XMLDataProcessor extends DataProcessor {
    private data: string = '';

    protected readData(): void {
        console.log('Reading XML file...');
        this.data = '<users><user><name>John</name><age>30</age></user></users>';
    }
}
```

```
}

protected validateData(): void {
  console.log('Validating XML structure...');
  console.log('XML structure is valid');
}

protected transformData(): void {
  console.log('Transforming XML to CSV...');
  // Simulate transformation
  this.data = 'name,age\nJohn,30';
}

protected saveData(): void {
  console.log('Saving CSV file...');
  console.log(`Data saved: ${this.data}`);
}
}

// Example 2: Report Generation
// Shows how Template Method can be used for report generation

abstract class ReportGenerator {
  // Template method
  generateReport(): void {
    console.log('\n=== Generating Report ===');
    this.collectData();
    this.analyzeData();
    this.formatReport();
    this.exportReport();
    console.log('=== Report Generated ===');
  }

  protected abstract collectData(): void;
  protected abstract analyzeData(): void;
  protected abstract formatReport(): void;
  protected abstract exportReport(): void;
}

class SalesReport extends ReportGenerator {
  protected collectData(): void {
    console.log('Collecting sales data from database...');
  }

  protected analyzeData(): void {
    console.log('Analyzing sales trends...');
    console.log('Sales increased by 15% this quarter');
  }

  protected formatReport(): void {
    console.log('Formatting sales report with charts...');
  }

  protected exportReport(): void {
```

```
        console.log('Exporting to PDF...');
    }
}

class InventoryReport extends ReportGenerator {
    protected collectData(): void {
        console.log('Collecting inventory data...');
    }

    protected analyzeData(): void {
        console.log('Analyzing stock levels...');
        console.log('3 items need restocking');
    }

    protected formatReport(): void {
        console.log('Formatting inventory report with tables...');
    }

    protected exportReport(): void {
        console.log('Exporting to Excel...');
    }
}

// Example 3: Game Character Creation
// Shows how Template Method can be used for character creation

abstract class GameCharacter {
    // Template method
    createCharacter(): void {
        console.log('\n=== Creating Character ===');
        this.chooseName();
        this.selectRace();
        this.selectClass();
        this.assignStats();
        this.equipStartingItems();
        console.log('=== Character Created ===');
    }

    protected abstract chooseName(): void;
    protected abstract selectRace(): void;
    protected abstract selectClass(): void;
    protected abstract assignStats(): void;
    protected abstract equipStartingItems(): void;

    // Hook method
    protected displayCharacter(): void {
        console.log('Character ready for adventure!');
    }
}

class Warrior extends GameCharacter {
    protected chooseName(): void {
        console.log('Name chosen: Thorin');
    }
}
```

```
protected selectRace(): void {
    console.log('Race selected: Dwarf');
}

protected selectClass(): void {
    console.log('Class selected: Warrior');
}

protected assignStats(): void {
    console.log('Stats assigned: STR: 18, DEX: 12, CON: 16, INT: 8, WIS:
10');
}

protected equipStartingItems(): void {
    console.log('Equipped: Iron Sword, Steel Shield, Chain Mail');
}

protected displayCharacter(): void {
    super.displayCharacter();
    console.log('Special ability: Battle Rage');
}
}

class Mage extends GameCharacter {
    protected chooseName(): void {
        console.log('Name chosen: Gandalf');
    }

    protected selectRace(): void {
        console.log('Race selected: Human');
    }

    protected selectClass(): void {
        console.log('Class selected: Mage');
    }

    protected assignStats(): void {
        console.log('Stats assigned: STR: 8, DEX: 14, CON: 10, INT: 18, WIS:
16');
    }

    protected equipStartingItems(): void {
        console.log('Equipped: Wooden Staff, Robe, Spellbook');
    }

    protected displayCharacter(): void {
        super.displayCharacter();
        console.log('Special ability: Fireball');
    }
}

class Rogue extends GameCharacter {
    protected chooseName(): void {
```

```
        console.log('Name chosen: Legolas');
    }

    protected selectRace(): void {
        console.log('Race selected: Elf');
    }

    protected selectClass(): void {
        console.log('Class selected: Rogue');
    }

    protected assignStats(): void {
        console.log('Stats assigned: STR: 12, DEX: 18, CON: 12, INT: 14, WIS: 12');
    }

    protected equipStartingItems(): void {
        console.log('Equipped: Dagger, Leather Armor, Lockpicks');
    }

    protected displayCharacter(): void {
        super.displayCharacter();
        console.log('Special ability: Stealth');
    }
}

// Demo code
console.log('=== Template Method Pattern Demo ===\n');

// Example 1: Data Processing
console.log('--- Data Processing Example ---');

const csvProcessor = new CSVDataProcessor();
csvProcessor.processData();

const xmlProcessor = new XMLDataProcessor();
xmlProcessor.processData();

// Example 2: Report Generation
console.log('\n--- Report Generation Example ---');

const salesReport = new SalesReport();
salesReport.generateReport();

const inventoryReport = new InventoryReport();
inventoryReport.generateReport();

// Example 3: Game Character Creation
console.log('\n--- Game Character Creation Example ---');

const warrior = new Warrior();
warrior.createCharacter();

const mage = new Mage();
```

```
mage.createCharacter();

const rogue = new Rogue();
rogue.createCharacter();
```

Visitor

Description: Lets you **add new operations to existing object structures** without modifying the objects. A visitor object carries the operation to be performed.

- **Why it is used:** To separate an algorithm from the object structure it operates on, making it easy to add new operations without touching existing classes.
- **Where it is commonly used:** Compilers (AST traversal), document export systems (export to PDF, HTML, XML), tax/discount calculation systems.

Summary Table

Category	Focus	Goal
Creational	Object creation	Control how objects are instantiated
Structural	Object composition	Build flexible and efficient structures
Behavioral	Object interaction	Define clear communication between objects

Tip for Beginners: Don't try to memorize all patterns at once. Start with the most commonly used ones — **Singleton**, **Factory Method**, **Observer**, **Strategy**, and **Decorator** — and learn the rest as you encounter real-world problems they solve.

Code Example (JAVASCRIPT)

```
/*
 * VISITOR DESIGN PATTERN
 *
 * WHAT IS IT?
 * The Visitor pattern lets you define a new operation without changing the
 * classes of the elements on which it operates.
 * It separates the algorithm from the object structure it operates on.
 *
 * WHY DOES IT EXIST?
 * - When you need to add operations to a complex object structure without
 *   changing the structure
 * - When you want to perform operations across different object types
 * - When you want to centralize related operations in a single class
 *
 * WHEN TO USE IT?
 * - When you have a complex object structure with different types of
 *   elements
 * - When you need to perform many unrelated operations on these elements
```

```

* - When the object structure is stable but you want to add new operations frequently
* - Common use cases: document processing, compiler AST traversal, file system operations, shopping cart calculations
*
* PROS:
* - Open/Closed Principle (can add new operations without changing element classes)
* - Single Responsibility Principle (moves related operations to a single visitor)
* - Can accumulate state while traversing the object structure
* - Works well with Composite pattern
*
* CONS:
* - Adding new element classes requires updating all visitors
* - Can violate encapsulation by exposing element internals
* - Can lead to complex visitor hierarchies
* - May make the code harder to understand
*
* REAL-WORLD ANALOGY:
* Think of a tax inspector visiting different types of businesses. The inspector (visitor) visits a restaurant, a retail store, and a factory (elements). Each business type calculates taxes differently. The inspector knows how to calculate taxes for each type but doesn't change how the businesses operate. You can add new types of inspections (health inspector, fire inspector) without changing the businesses.
*/

// Visitor interface - declares visit methods for each concrete element
interface Visitor {
  visitBook(book: Book): void;
  visitFruit(fruit: Fruit): void;
  visitElectronic(electronic: Electronic): void;
}

// Element interface - declares the accept method
interface Item {
  accept(visitor: Visitor): void;
  getPrice(): number;
}

// Concrete Element 1: Book
class Book implements Item {
  private title: string;
  private author: string;
  private price: number;

  constructor(title: string, author: string, price: number) {
    this.title = title;
    this.author = author;
    this.price = price;
  }

  accept(visitor: Visitor): void {

```

```
        visitor.visitBook(this);
    }

    getPrice(): number {
        return this.price;
    }

    getTitle(): string { return this.title; }
    getAuthor(): string { return this.author; }
}

// Concrete Element 2: Fruit
class Fruit implements Item {
    private name: string;
    private price: number;
    private weight: number;

    constructor(name: string, price: number, weight: number) {
        this.name = name;
        this.price = price;
        this.weight = weight;
    }

    accept(visitor: Visitor): void {
        visitor.visitFruit(this);
    }

    getPrice(): number {
        return this.price;
    }

    getName(): string { return this.name; }
    getWeight(): number { return this.weight; }
}

// Concrete Element 3: Electronic
class Electronic implements Item {
    private model: string;
    private brand: string;
    private price: number;

    constructor(brand: string, model: string, price: number) {
        this.brand = brand;
        this.model = model;
        this.price = price;
    }

    accept(visitor: Visitor): void {
        visitor.visitElectronic(this);
    }

    getPrice(): number {
        return this.price;
    }
}
```

```
    getModel(): string { return this.model; }
    getBrand(): string { return this.brand; }
}

// Concrete Visitor 1: Price Calculator
class PriceCalculator implements Visitor {
    private totalPrice: number = 0;

    visitBook(book: Book): void {
        this.totalPrice += book.getPrice();
        console.log(`Book: ${book.getTitle()} - ${book.getPrice()}`);
    }

    visitFruit(fruit: Fruit): void {
        this.totalPrice += fruit.getPrice();
        console.log(`Fruit: ${fruit.getName()} - ${fruit.getPrice()}`);
    }

    visitElectronic(electronic: Electronic): void {
        this.totalPrice += electronic.getPrice();
        console.log(`Electronic: ${electronic.getBrand()}
${electronic.getModel()} - ${electronic.getPrice()}`);
    }

    getTotalPrice(): number {
        return this.totalPrice;
    }
}

// Concrete Visitor 2: Description Generator
class DescriptionGenerator implements Visitor {
    visitBook(book: Book): void {
        console.log(`Book: "${book.getTitle()}" by ${book.getAuthor()} - A
great read!`);
    }

    visitFruit(fruit: Fruit): void {
        console.log(`Fruit: ${fruit.getName()} (${fruit.getWeight()}kg) - Fresh
and healthy!`);
    }

    visitElectronic(electronic: Electronic): void {
        console.log(`Electronic: ${electronic.getBrand()}
${electronic.getModel()} - High-tech gadget!`);
    }
}

// Concrete Visitor 3: Tax Calculator
class TaxCalculator implements Visitor {
    private totalTax: number = 0;

    visitBook(book: Book): void {
        const tax = book.getPrice() * 0.05; // 5% tax on books
```

```
        this.totalTax += tax;
        console.log(`Book tax: ${tax}`);
    }

    visitFruit(fruit: Fruit): void {
        const tax = fruit.getPrice() * 0.02; // 2% tax on food
        this.totalTax += tax;
        console.log(`Fruit tax: ${tax}`);
    }

    visitElectronic(electronic: Electronic): void {
        const tax = electronic.getPrice() * 0.10; // 10% tax on electronics
        this.totalTax += tax;
        console.log(`Electronic tax: ${tax}`);
    }

    getTotalTax(): number {
        return this.totalTax;
    }
}

// Object Structure - manages the elements
class ShoppingCart {
    private items: Item[] = [];

    addItem(item: Item): void {
        this.items.push(item);
    }

    acceptVisitor(visitor: Visitor): void {
        for (const item of this.items) {
            item.accept(visitor);
        }
    }
}

// Example 2: File System Visitor
// Shows how Visitor can be used for file system operations

interface FileSystemVisitor {
    visitFile(file: File): void;
    visitDirectory(directory: Directory): void;
}

interface FileSystemNode {
    accept(visitor: FileSystemVisitor): void;
    getName(): string;
    getSize(): number;
}

class File implements FileSystemNode {
    private name: string;
    private size: number;
```

```
    constructor(name: string, size: number) {
        this.name = name;
        this.size = size;
    }

    accept(visitor: FileSystemVisitor): void {
        visitor.visitFile(this);
    }

    getName(): string { return this.name; }
    getSize(): number { return this.size; }
}

class Directory implements FileSystemNode {
    private name: string;
    private children: FileSystemNode[] = [];

    constructor(name: string) {
        this.name = name;
    }

    addChild(child: FileSystemNode): void {
        this.children.push(child);
    }

    accept(visitor: FileSystemVisitor): void {
        visitor.visitDirectory(this);
    }

    getName(): string { return this.name; }
    getSize(): number {
        return this.children.reduce((total, child) => total + child.getSize(),
0);
    }
}

class SizeCalculator implements FileSystemVisitor {
    private totalSize: number = 0;

    visitFile(file: File): void {
        this.totalSize += file.getSize();
        console.log(`File: ${file.getName()} - ${file.getSize()} bytes`);
    }

    visitDirectory(directory: Directory): void {
        console.log(`Directory: ${directory.getName()} - ${directory.getSize()}
bytes`);
        this.totalSize += directory.getSize();
    }

    getTotalSize(): number {
        return this.totalSize;
    }
}
```

```
class XMLExporter implements FileSystemVisitor {
    visitFile(file: File): void {
        console.log(` <file name="${file.getName()}"
size="${file.getSize()}" />`);
    }

    visitDirectory(directory: Directory): void {
        console.log(` <directory name="${directory.getName()}"
size="${directory.getSize()}" >`);
    }
}

// Example 3: Employee Salary Calculator
// Shows how Visitor can be used for salary calculations

interface SalaryVisitor {
    visitEmployee(employee: Employee): void;
    visitManager(manager: Manager): void;
    visitDeveloper(developer: Developer): void;
    visitDesigner(designer: Designer): void;
}

interface Employee {
    accept(visitor: SalaryVisitor): void;
    getName(): string;
    getBaseSalary(): number;
}

class Manager implements Employee {
    private name: string;
    private baseSalary: number;
    private bonus: number;

    constructor(name: string, salary: number, bonus: number) {
        this.name = name;
        this.baseSalary = salary;
        this.bonus = bonus;
    }

    accept(visitor: SalaryVisitor): void {
        visitor.visitManager(this);
    }

    getName(): string { return this.name; }
    getBaseSalary(): number { return this.baseSalary; }
    getBonus(): number { return this.bonus; }
}

class Developer implements Employee {
    private name: string;
    private baseSalary: number;
    private overtimeHours: number;
```

```
    constructor(name: string, salary: number, overtime: number) {
        this.name = name;
        this.baseSalary = salary;
        this.overtimeHours = overtime;
    }

    accept(visitor: SalaryVisitor): void {
        visitor.visitDeveloper(this);
    }

    getName(): string { return this.name; }
    getBaseSalary(): number { return this.baseSalary; }
    getOvertimeHours(): number { return this.overtimeHours; }
}

class Designer implements Employee {
    private name: string;
    private baseSalary: number;
    private projectsCompleted: number;

    constructor(name: string, salary: number, projects: number) {
        this.name = name;
        this.baseSalary = salary;
        this.projectsCompleted = projects;
    }

    accept(visitor: SalaryVisitor): void {
        visitor.visitDesigner(this);
    }

    getName(): string { return this.name; }
    getBaseSalary(): number { return this.baseSalary; }
    getProjectsCompleted(): number { return this.projectsCompleted; }
}

class BaseEmployee implements Employee {
    private name: string;
    private baseSalary: number;

    constructor(name: string, salary: number) {
        this.name = name;
        this.baseSalary = salary;
    }

    accept(visitor: SalaryVisitor): void {
        visitor.visitEmployee(this);
    }

    getName(): string { return this.name; }
    getBaseSalary(): number { return this.baseSalary; }
}

class SalaryCalculator implements SalaryVisitor {
    private totalSalary: number = 0;
```

```
visitEmployee(employee: Employee): void {
    const salary = employee.getBaseSalary();
    this.totalSalary += salary;
    console.log(`Employee: ${employee.getName()} - ${salary}`);
}

visitManager(manager: Manager): void {
    const salary = manager.getBaseSalary() + manager.getBonus();
    this.totalSalary += salary;
    console.log(`Manager: ${manager.getName()} - ${salary} (base:
${manager.getBaseSalary()} + bonus: ${manager.getBonus()})`);
}

visitDeveloper(developer: Developer): void {
    const overtimePay = developer.getOvertimeHours() * 50.0;
    const salary = developer.getBaseSalary() + overtimePay;
    this.totalSalary += salary;
    console.log(`Developer: ${developer.getName()} - ${salary} (base:
${developer.getBaseSalary()} + overtime: ${overtimePay})`);
}

visitDesigner(designer: Designer): void {
    const projectBonus = designer.getProjectsCompleted() * 500.0;
    const salary = designer.getBaseSalary() + projectBonus;
    this.totalSalary += salary;
    console.log(`Designer: ${designer.getName()} - ${salary} (base:
${designer.getBaseSalary()} + project bonus: ${projectBonus})`);
}

getTotalSalary(): number {
    return this.totalSalary;
}
}

// Demo code
console.log('=== Visitor Pattern Demo ===\n');

// Example 1: Shopping Cart
console.log('--- Shopping Cart Example ---');

const cart = new ShoppingCart();
cart.addItem(new Book('Design Patterns', 'Erich Gamma', 49.99));
cart.addItem(new Fruit('Apple', 1.99, 0.5));
cart.addItem(new Electronic('Apple', 'iPhone 15', 999.99));

console.log('\nCalculating total price:');
const priceCalc = new PriceCalculator();
cart.acceptVisitor(priceCalc);
console.log(`Total: ${priceCalc.getTotalPrice()}`);

console.log('\nGenerating descriptions:');
const descGen = new DescriptionGenerator();
cart.acceptVisitor(descGen);
```

```
console.log('\nCalculating tax:');
const taxCalc = new TaxCalculator();
cart.acceptVisitor(taxCalc);
console.log(`Total tax: ${taxCalc.getTotalTax()}`);

// Example 2: File System
console.log('\n--- File System Example ---');

const root = new Directory('root');
const file1 = new File('document.txt', 1024);
const file2 = new File('image.jpg', 2048);
const subdir = new Directory('subdir');
const file3 = new File('data.csv', 512);

subdir.addChild(file3);
root.addChild(file1);
root.addChild(file2);
root.addChild(subdir);

console.log('\nCalculating sizes:');
const sizeCalc = new SizeCalculator();
root.accept(sizeCalc);
console.log(`Total size: ${sizeCalc.getTotalSize()} bytes`);

console.log('\nExporting to XML:');
console.log('<filesystem>');
const xmlExporter = new XMLExporter();
root.accept(xmlExporter);
console.log('</filesystem>');

// Example 3: Employee Salary
console.log('\n--- Employee Salary Example ---');

const employees: Employee[] = [
  new Manager('Alice', 80000, 20000),
  new Developer('Bob', 70000, 10),
  new Designer('Charlie', 65000, 5)
];

console.log('\nCalculating total salary:');
const salaryCalc = new SalaryCalculator();
for (const emp of employees) {
  emp.accept(salaryCalc);
}
console.log(`Total salary: ${salaryCalc.getTotalSalary()}`);
```